Recursive Functions

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Exercise

Write a Python function called countdown that takes a positive integer n as input and prints a countdown from n to zero.

If the input number is negative, display an error message.

To implement the function, you must exclusively use a while loop and the parameter n passed as input to the function.

Declaring any additional variables inside the function is not allowed.

Then, call the function with n = -5 and n = 5.

Expected Output:

```
Error! Inserted number is negative!
-----------------------------------------------
5
4
3
2
1
0
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Solution

Your code might look like this:

```
def countdown(n:int) -> None:
    # n is positive (n >= 0)
    if n >=0 :
        # print the countdown, while n>=0
        while n>=0:
            print(n)
            n = n - 1

    # n is negative (n < 0)
    else:
        print("Error! Inserted number is negative!")


# calling function countdown for n = -5
countdown(-5)


# calling function countdown for n = 5
countdown(5)
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions - Introduction

The programs examined so far have generally been structured as functions that call each other in a disciplined and hierarchical manner.

For certain types of problems, it is useful to have functions that call themselves.

A function that calls itself is called a **recursive function**.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Exercise

Rewrite the introductory exercise by transforming the countdown function into a recursive function.

Expected Output:

Error! Inserted number is negative!

5

4

3

2

1

0

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions /1

Recursive Method

5

```
countdown(5)
```

4

```
countdown(4)
```

3

```
countdown(3)
```

2

```
countdown(2)
```

1

```
countdown(1)
```

0

```
countdown(0)
```

Iterative Method

`n = 5`	`Iter1: 5`
`n = 4`	`Iter2: 4`
`n = 3`	`Iter3: 3`
`n = 2`	`Iter4: 2`
`n = 1`	`Iter5: 1`
`n = 0`	`Iter6: 0`

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions /2

First, let's consider the simplest case of the problem to be solved. This is the easiest case to handle and/or the case where *n* takes the smallest possible values within the allowed range.

In our example, the simplest case to handle is when *n < 0*, in which case the function simply prints an error message on the screen.

```
def countdown(n:int) -> None:

    # n is negative
    if n < 0 :
        print("Error! Inserted number is negative!")
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions /3

Just as in the iterative method we need to impose a condition that stops the repetition of the loop, we must do the same in the recursive method. We need to find a condition such that, when satisfied, the recursion stops.

In our example, we need to print the numbers from *n* down to *0*, inclusive.
Therefore, we want our recursion to end when *n = 0*. So, if *n = 0*, we want our function to print *0* on the screen.

```
def countdown(n:int) -> None:

    # n is negative
    if n < 0 :
        print("Error! Inserted number is negative!")

    # n = 0 must stop the recursive process
    elif n == 0:
        print(0)
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions /4

The case where *n* is less than 0 and the case where *n* equals 0 are situations in which the value of *n* allows the recursion to stop. For this reason, these cases are called **base cases** or **stopping cases**, as the recursive process will not require further recursive calls.

Thus, when a stopping case occurs, the recursion terminates.

In our example, the recursion must stop if $n = 0$, or it should not start at all if $n < 0$.

```
def countdown(n:int) -> None:

    # n is negative
    if n < 0 :
        print("Error! Inserted number is negative!")

    # n = 0 must stop the recursive process
    elif n == 0:
        print(0)
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions /5

What happens in all other cases? That is, what happens if the value of *n* is between 5 and 1?

To solve these cases recursively, it is necessary to reduce the problem to a simpler version of itself.

This means reformulating the problem in terms of a call to the *countdown* function.

For example, if *n = 5*, we want to call the function *countdown(5)*, passing 5 as the argument, and print the number 5 on the screen. However, we still need to print the numbers 4, 3, 2, 1, and 0.

How can we achieve this?

- Since 5 is greater than 0, it is not negative, so the first *if* condition is skipped.
- The first *elif* is also skipped because 5 is not equal to 0.

```
def countdown(n:int) -> None:

    # n is negative
    if n < 0 :
        print("Error! Inserted number is negative!")

    # n = 0 must stop the recursive process
    elif n == 0:
        print(0)
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions /6

Therefore, we want our function, in all cases other than the base cases, to:

- For *n = 5,* since 5 is greater than 0 and not equal to 0, the *countdown* function should:

- Print the number 5 on the screen.
- Call the function *countdown(4)* to print the number 4.

For *n = 4,* since 4 is greater than 0 and not equal to 0, the *countdown* function should:

- Print the number 4 on the screen.
- Call the function *countdown(3)* to print the number 3.

And so on.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions /7

Until we reach *n = 1*, where since 1 is positive and 1 is not equal to 0, the *countdown* function should:

- Print *1* on the screen.
- Call the function *countdown(0)* to print *0* on the screen.

Now, when *countdown(0)* is called, *n* is equal to 0, so the condition in the *elif* statement is checked.
Thus, if *n = 0*:

- Print *0* on the screen.
- Stop the recursive process.

```
def countdown(n:int) -> None:

    # n is negative
    if n < 0 :
        print("Error! Inserted number is negative!")

    # n = 0 must stop the recursive process
    elif n == 0:
        print(0)
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions /8

Let's go back to *n = 5*.

So, if *n = 5*, 5 is positive and 5 is not equal to 0. Therefore, the function *countdown(5)* must:

- Print the number *5* on the screen.
- Call the function *countdown(4)*.

If *n = 5*, how can I write *countdown(4)* in terms of *n*?

Answer: ***countdown(n - 1)***.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions /9

We can now write our recursive *countdown* function.

```
def countdown(n:int) -> None:

    # n is negative
    if n < 0 :
        print("Error! Inserted number is negative!")

    # n = 0 must stop the recursive process
    elif n == 0:
        print(0)

    # other cases
    else:
        print(n)
        countdown(n-1)
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions /10

Testing the recursive countdown function.

```
countdown(-5)
```

```
>>> Error! Inserted number is negative!
```

```
countdown(0)
```

```
>>> 0
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions /11

Testing the recursive countdown function.

```
countdown(5)
```

```
>>>
5
4
3
2
1
0
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions /12

What if we omit the *elif* statement where we consider the case *n = 0*?

```
def countdown(n:int) -> None:

    # n is negative
    if n < 0 :
        print("Error! Inserted number is negative!")
    # other cases
    else:
        print(n)
        countdown(n-1)

countdown(5)
```

```
>>>
5
4
3
2
1
0
Error! Inserted number is negative!
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions /13

What if we omit the *if* statement where we consider the case *n < 0* and *elif* statement for case *n = 0* ?

```
def countdown(n:int) -> None:

    print(n)
    countdown(n-1)

countdown(5)
```

```
>>>
5
4
3
2
1
0
-1
-2
-3
-4
-5
-6
...
```

Infinite Recursion!

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions

So, as we have seen in the example case, a recursive function calls itself, meaning that in its definition, the function contains a call to itself.

3

```
def countdown(n:int) -> None:
    if n < 0 :
        print("Error!")
    elif n == 0:
        print(0)
    else:
        print(n)
        countdown(n-1)
```

2

```
def countdown(n:int) -> None:
    if n < 0 :
        print("Error!")
    elif n == 0:
        print(0)
    else:
        print(n)
        countdown(n-1)
```

1

```
def countdown(n:int) -> None:
    if n < 0 :
        print("Error!")
    elif n == 0:
        print(0)
    else:
        print(n)
        countdown(n-1)
```

0

```
def countdown(n:int) -> None:
    if n < 0 :
        print("Error!")
    elif n == 0:
        print(0)
    else:
        print(n)
        countdown(n-1)
```

```
>>>
    3
    2
    1
    0
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 1- Iterative/1

Write a function called *sum* that takes a positive integer number *n* as input and returns the sum of the numbers from *0* to *n*.
If the input number *n* is negative, display an error message and the function must return *0*.

To implement the *sum* function, you must exclusively use a while loop and the parameter *n* passed as input to the function.

It is allowed to declare only one variable inside the function to manage the sum.

Then, call the function sum for *n = -5* and *n = 5*.

Expected Output:

Error! Inserted number is negative!

0

15

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 1- Iterative/2

Your code might look like this:

```
def sum(n:int) -> int:

    # n is negative
    if n < 0:

        print("Error! Inserted number is negative!")
        return 0

    # n is positive
    else:

        # define a sum
        sum:int = 0

        # compute sum until n >= 0
        while n>=0:
            sum = sum + n
            n = n-1

        # return sum as int
        return int(sum)
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 1- Iterative/3

Your code might look like this:

```
# calling sum function for n = -5
print(sum(-5))

>>>
Error! Inserted number is negative!
0
```

```
# calling sum function for n = 5
print(sum(5))

>>>
15
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 1- Recursive/1

Write a Python function called *recursiveSum* that, given an integer *n* as input, recursively calculates the sum of the numbers from *0* to *n*.

If the input number *n* is negative, display an error message and the function must return *0*.

Then, call the function recursiveSum for *n = -5* and *n = 5*.

Expected Output:

```
Error! Inserted number is negative!
0
-----------------------------------------------
15
```

Recursive Functions – Example 1- Recursive/2

First, let's consider the simplest case of the problem to be solved. This is the easiest case to handle and/or the case where *n* takes the smallest possible values within the allowed range.

In our example, the simplest case to handle is when *n < 0*, in which case the function simply prints an error message on the screen.

```
def recursiveSum(n:int) -> int:

    # n is negative
    if n < 0:
        print("Error! Inserted number is negative!")
        return 0
```

Recursive Functions – Example 1- Recursive/3

Just as in the iterative method we need to impose a condition that stops the repetition of the loop, we must do the same in the recursive method. We need to find a condition such that, when satisfied, the recursion stops.

In our example, we need to sum the numbers from *n* down to *0*, inclusive.
Therefore, we want our recursion to end when *n = 0*. So, if *n = 0*, we want our function to return *0* .

```
def recursiveSum(n:int) -> int:

    # n is negative
    if n < 0:
        print("Error! Inserted number is negative!")
        return None

    # if n = 0 recursive process must be stopped
    elif n == 0:
        return 0
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 1- Recursive/4

```
recursiveSum(5)
```

↓

```
recursiveSum(4)
```

↓

```
recursiveSum(3)
```

↓

```
recursiveSum(2)
```

↓

```
recursiveSum(1)
```

↓

```
recursiveSum(0)
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 1- Recursive/4

```
rS(5)        (5>0, 5!=0)
```

↓

```
5  + rS(4)
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 1- Recursive/4

```
rS(5)          (5>0, 5!=0)
  5 + rS(4)        (4>0, 4!=0)
      4 + rS(3)
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 1- Recursive/4

```
rS(5)          (5>0, 5!=0)
  5 + rS(4)        (4>0, 4!=0)
    4 + rS(3)          (3>0, 3!=0)
      3 + rS(2)
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 1- Recursive/4

```
rS(5)          (5>0, 5!=0)
  5 + rS(4)        (4>0, 4!=0)
    4 + rS(3)        (3>0, 3!=0)
      3 + rS(2)        (2>0, 2!=0)
        2 + rS(1)
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 1- Recursive/4

```
rS(5)          (5>0, 5!=0)
  5 + rS(4)        (4>0, 4!=0)
    4 + rS(3)        (3>0, 3!=0)
      3 + rS(2)        (2>0, 2!=0)
        2 + rS(1)        (1>0, 1!=0)
          1 + rS(0)
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 1- Recursive/4

```
rS(5)          (5>0, 5!=0)
  5 + rS(4)        (4>0, 4!=0)
    4 + rS(3)        (3>0, 3!=0)
      3 + rS(2)        (2>0, 2!=0)
        2 + rS(1)        (1>0, 1!=0)
          1 + rS(0)        (0=0)
            0
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 1- Recursive/5

We can now write our recursive *recursive* function.

```
def recursiveSum(n:int) -> int:

    # n is negative
    if n < 0:
        print("Error! Inserted number is negative!")
        return None

    # if n = 0 recursive process must be stopped
    elif n == 0:
        return 0

    # if n is positive, compute recursive sum
    else:
        return int(n + recursiveSum(n-1))
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 1- Recursive/6

How *recursiveSum* function works? Earlier, we saw how to derive the recursive function *recursiveSum* following this scheme. Now, we need to understand how it calculates the sum.

```
rS(5)                    (5>0, 5!=0)
  5 + rS(4)              (4>0, 4!=0)
    4 + rS(3)            (3>0, 3!=0)
      3 + rS(2)          (2>0, 2!=0)
        2 + rS(1)        (1>0, 1!=0)
          1 + rS(0)      (0=0)
            0
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 1- Recursive/6

How *recursiveSum* function works?

```
1  + rS(0)
```

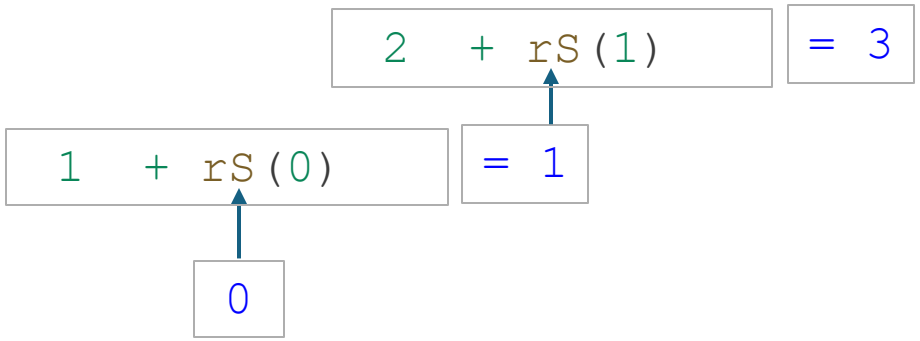
```
= 1
```

```
0
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

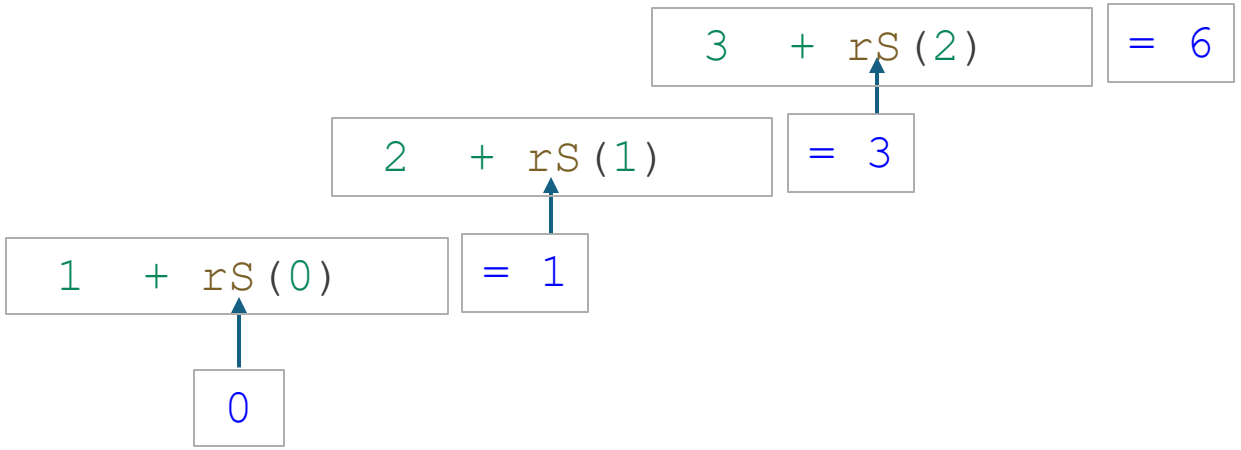
Recursive Functions – Example 1- Recursive/6

How *recursiveSum* function works?



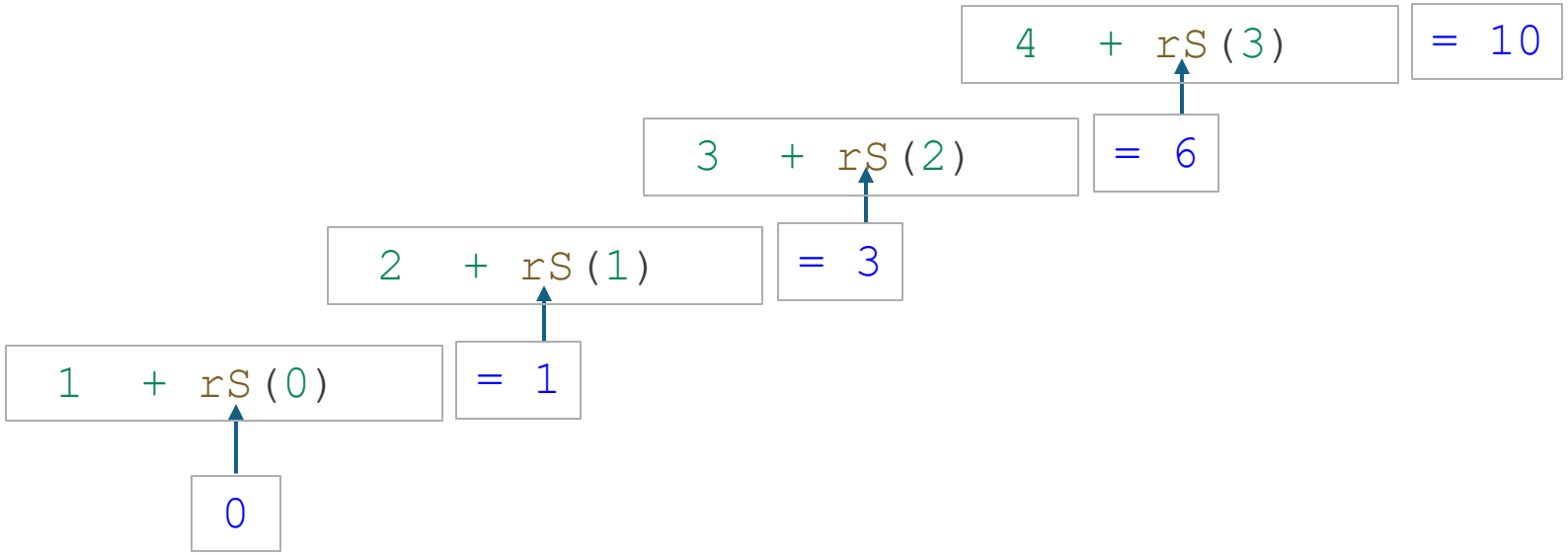
Recursive Functions – Example 1- Recursive/6

How *recursiveSum* function works?



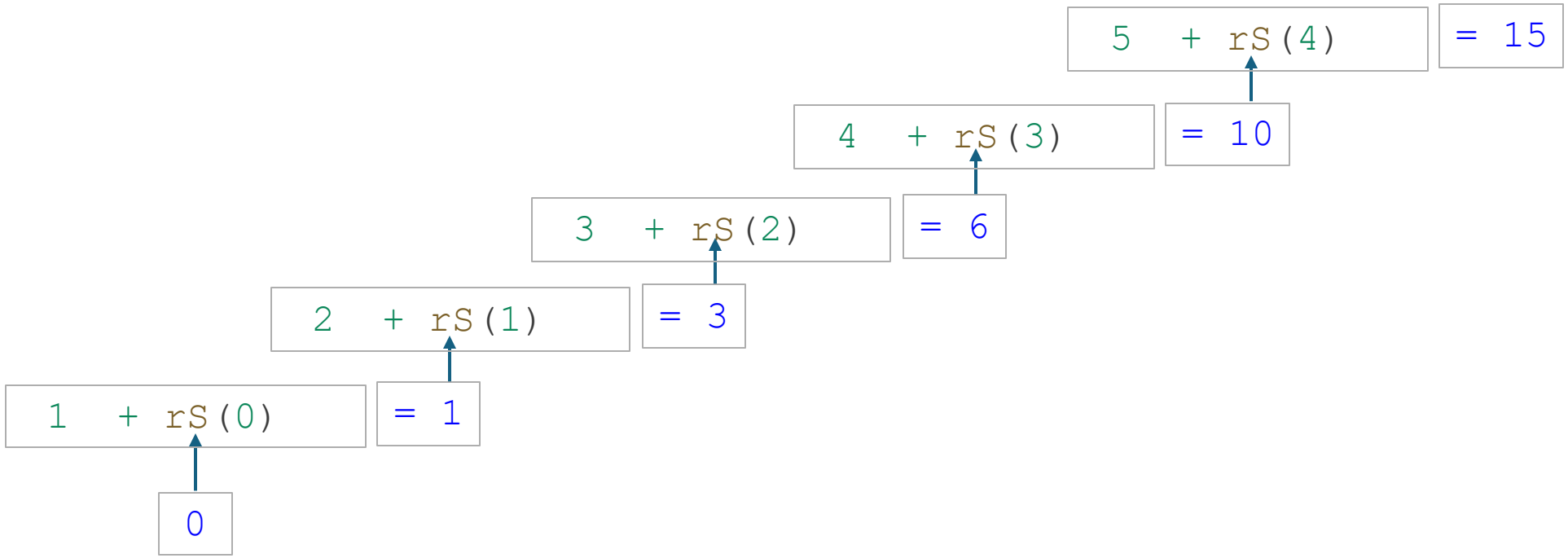
Recursive Functions – Example 1- Recursive/6

How *recursiveSum* function works?



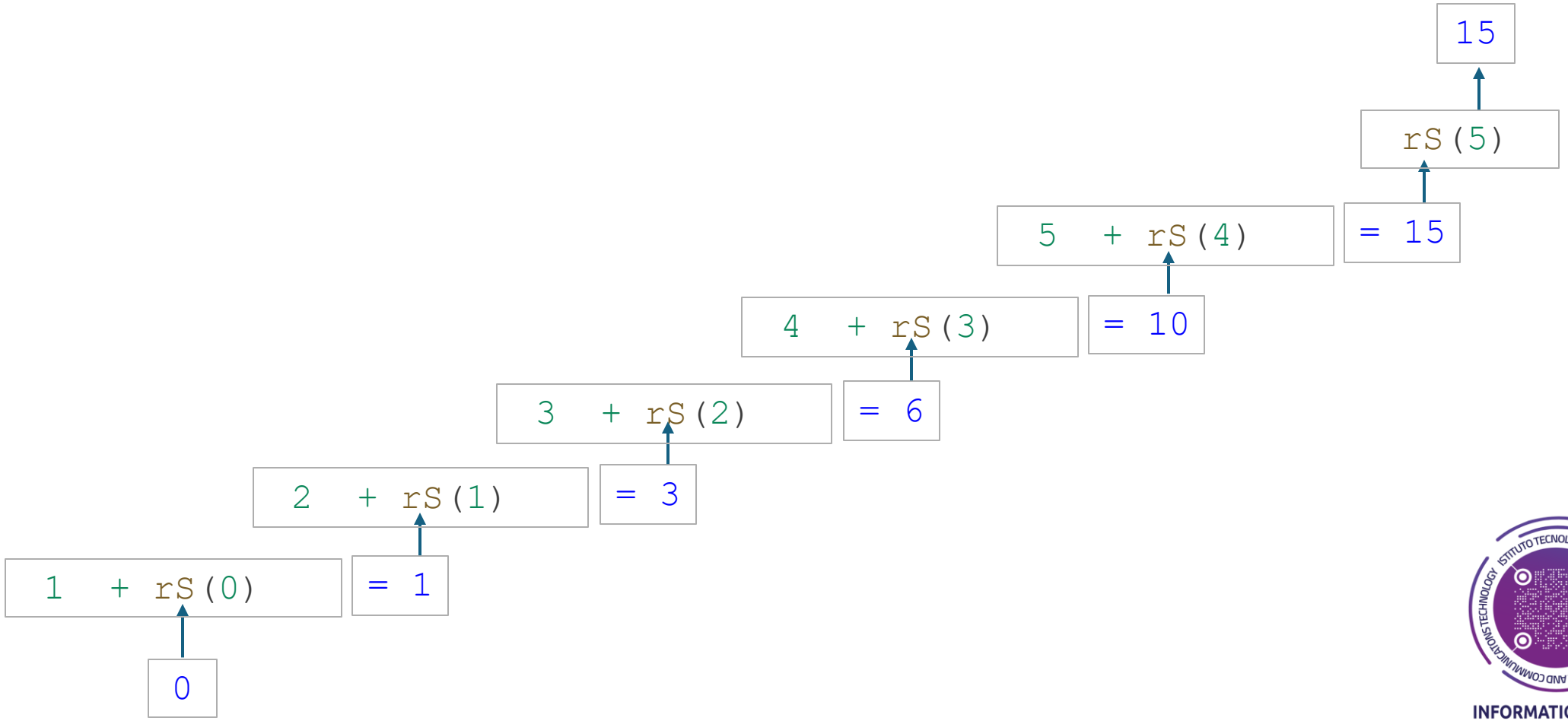
Recursive Functions – Example 1- Recursive/6

How *recursiveSum* function works?



Recursive Functions – Example 1- Recursive/6

How *recursiveSum* function works?



ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 1- Recursive/7

The example shows that when a recursive call to the function *recursiveSum* occurs, the execution is temporarily suspended because, in order to proceed and complete the operation, it is necessary to know the result of the recursive call.

Therefore, when a recursive call is encountered, for example, *rS(1)*, all the necessary information is saved to resume execution later. Then, the next recursive call, *rS(0)*, is executed.

Once *rS(0)* is completed, the outermost execution, *rS(1)*, is resumed.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY – INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 1- Recursive/8

Testing the *recursiveSum* function.

```
recursiveSum(-5)

>>> Error! Inserted number is negative!
None

recursiveSum(0)

>>> 0

recursiveSum(5)

>>>
15
```

Recursive Functions – Example 2- Iterative/1

Write a function *sumInRange* that calculates the sum of all integers between *a* and *b*, inclusive, where *a* and *b* are passed as input to the function.

To solve the exercise, it is mandatory to use a while loop, and it is assumed that the value of *b* is always greater than the value of *a*. Therefore, if *a* > *b*, it is necessary to swap the values to ensure that a is the smaller of the two.

Finally, it is allowed to declare only one variable, in addition to the function parameters, to store the sum.

Then, call the function *sumInRange* for *a = 5*, *b = 10* and for *a = 10*, *b = 5*.

Expected Output:

45

45

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 2- Iterative/2

Your code might look like this:

```
def sumInRange(a:int, b:int) -> int:

    # if a > b, swap values of a and b
    if a > b:

        # define a temporary variable called temp, containing value of a
        temp:int = a

        # swap values of a and b
        a = b
        b = temp # now b = a

    # define a sum
    sum:int = 0

    # compute sum until b = a
    while b>=a:

        sum = sum + b
        b = b - 1

    # return sum
    return int(sum)
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 2- Recursive/1

Write a recursive function recursiveSumInRange that calculates the sum of all integers between a and b, inclusive, where a and b are passed as input to the function.

Assume that the value of b is always greater than the value of a. Therefore, if a > b, it is necessary to swap the values to ensure that a is the smaller of the two.

Then, call the function recursiveSumInRange for a = 5, b = 10 and for a = 10, b = 5.

Expected Output:

45

45

Recursive Functions – Example 2- Recursive/2

First, let's consider the simplest case of the problem to be solved. We must swap values of a and b if a > b.

```
def recursiveSumInRange(a:int, b:int) -> int:

    # if a > b, swap values of a and b
    if a > b:

        # define a temporary variable called temp, containing value of a
        temp:int = a

        # swap values of a and b
        a = b
        b = temp # now b = a
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 2- Recursive/3

Just as in the iterative method we need to impose a condition that stops the repetition of the loop, we must do the same in the recursive method. We need to find a condition such that, when satisfied, the recursion stops.

In our example, we need to sum the numbers from *a* down to *b*, inclusive.
Therefore, we want our recursion to end when *b* = *a*. So, if *b* = *a*, we want our function to return *a* .

```
def recursiveSumInRange(a:int, b:int) -> int:

    # if a > b, swap values of a and b
    if a > b:

        # define a temporary variable called temp, containing value of a
        temp:int = a
        # swap values of a and b
        a = b
        b = temp # now b = a


    # if b = a, the recursive process must be stopped
    if b == a:
        return int(a)
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 2- Recursive/4

```
recursiveSumInRange(5,10)
```

↓

```
recursiveSumInRange(5,9)
```

↓

```
recursiveSumInRange(5,8)
```

↓

```
recursiveSumInRange(5,7)
```

↓

```
recursiveSumInRange(5,6)
```

↓

```
recursiveSumInRange(5,5)
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 2- Recursive/4

```
rSR(5,10)    (10!=5)
```

```
10  + rSR(5,9)
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 2- Recursive/4

```
rSR(5,10)        (10!=5)
    10  + rSR(5,9)        (9!=5)
            9  + rSR(5,8)
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 2- Recursive/4

```
rSR(5,10)        (10!=5)
    10  + rSR(5,9)        (9!=5)
            9  + rSR(5,8)        (8!=5)
                    8  + rSR(5,7)
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 2- Recursive/4

```
rSR(5,10)        (10!=5)
    10  + rSR(5,9)        (9!=5)
        9  + rSR(5,8)        (8!=5)
            8  + rSR(5,7)        (7!=5)
                7  + rSR(5,6)
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 2- Recursive/4

```
rSR(5,10)        (10!=5)
    10  + rSR(5,9)        (9!=5)
        9  + rSR(5,8)        (8!=5)
            8  + rSR(5,7)        (7!=5)
                7  + rSR(5,6)        (6!=5)
                    6  + rSR(5,5)
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 2- Recursive/4

```
rSR(5,10)        (10!=5)
  10  + rSR(5,9)        (9!=5)
    9  + rSR(5,8)        (8!=5)
      8  + rSR(5,7)        (7!=5)
        7  + rSR(5,6)        (6!=5)
          6  + rSR(5,5)        (5=5)
            5
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 2- Recursive/5

We can now write our recursive *recursive* function.

```
def recursiveSumInRange(a:int, b:int) -> int:

    # if a > b, swap values of a and b
    if a > b:

        # define a temporary variable called temp, containing value of a
        temp:int = a

        # swap values of a and b
        a = b
        b = temp # now b = a


    # if b = a, the recursive process must be stopped
    if b == a:

        return int(a)


    # otherwise, compute the sum recursively
    else:

        return int(b + recursiveSumInRange(a, b-1))
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 2- Recursive/6

How *recursiveSumInRange* function works?

```
6  + rSR(5,5)
```

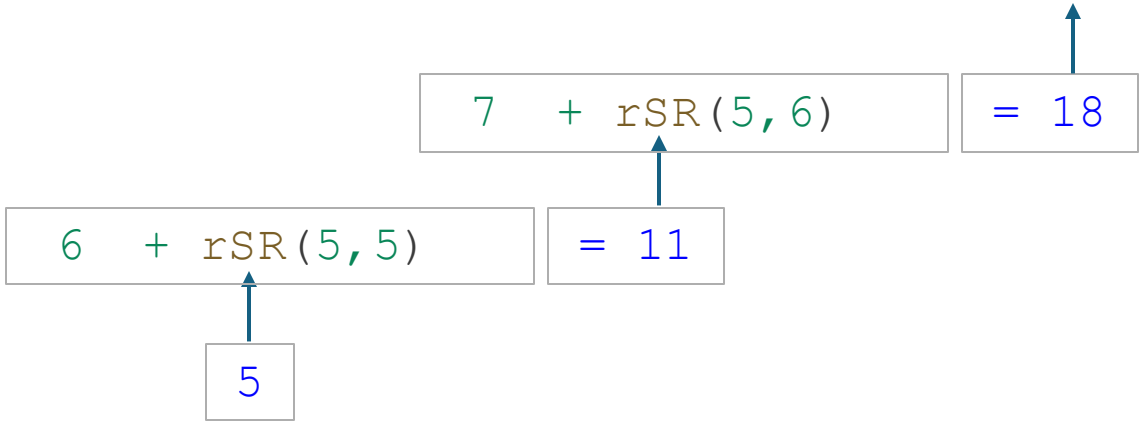
```
= 11
```

```
5
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

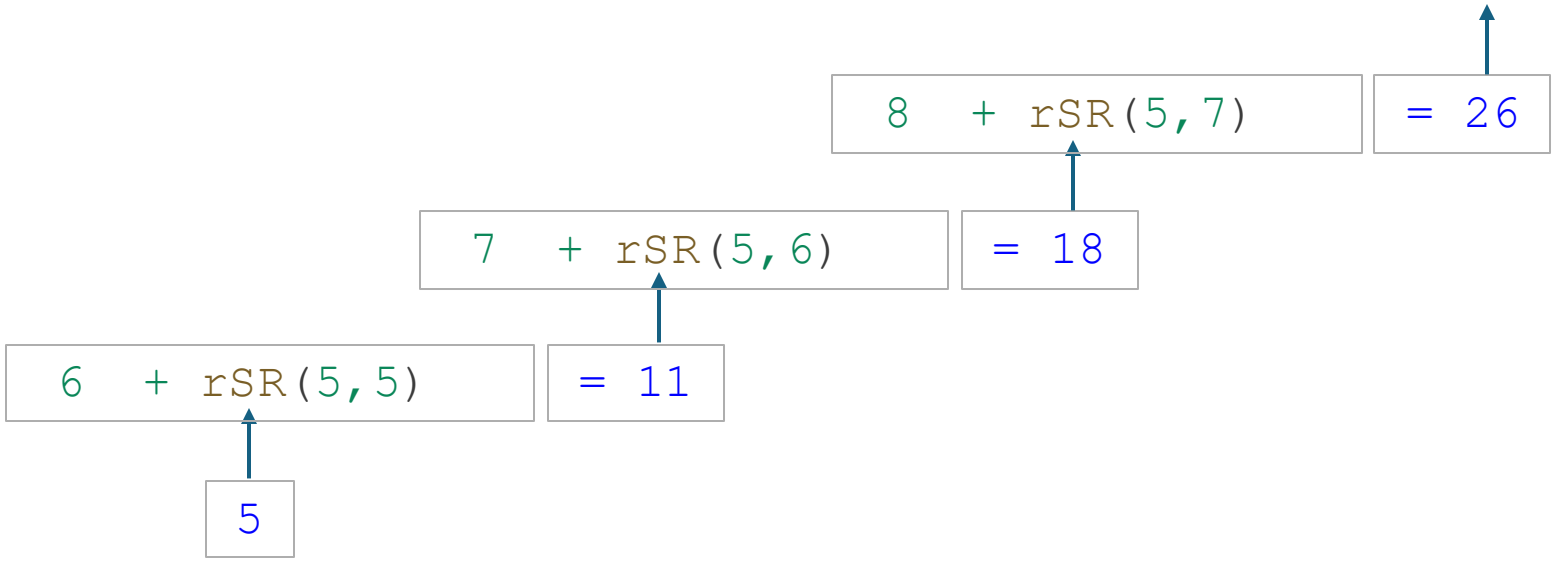
Recursive Functions – Example 2- Recursive/6

How *recursiveSumInRange* function works?



Recursive Functions – Example 2- Recursive/6

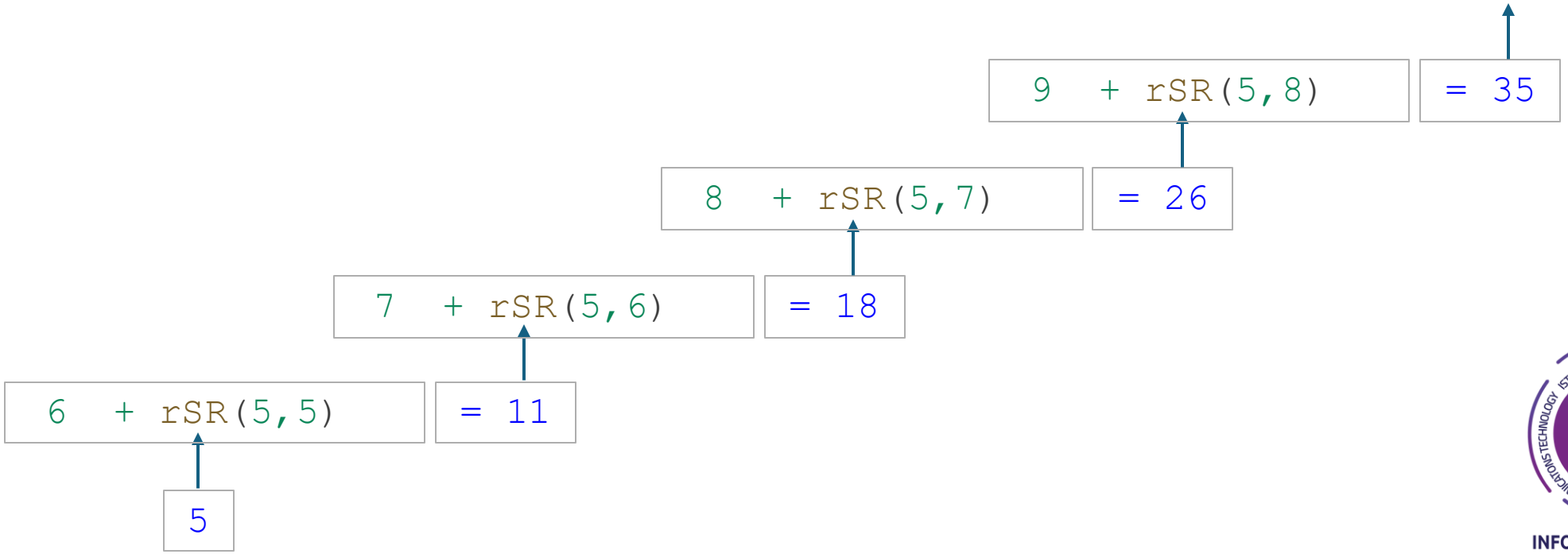
How *recursiveSumInRange* function works?



ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 2- Recursive/6

How *recursiveSumInRange* function works?



ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 2- Recursive/6

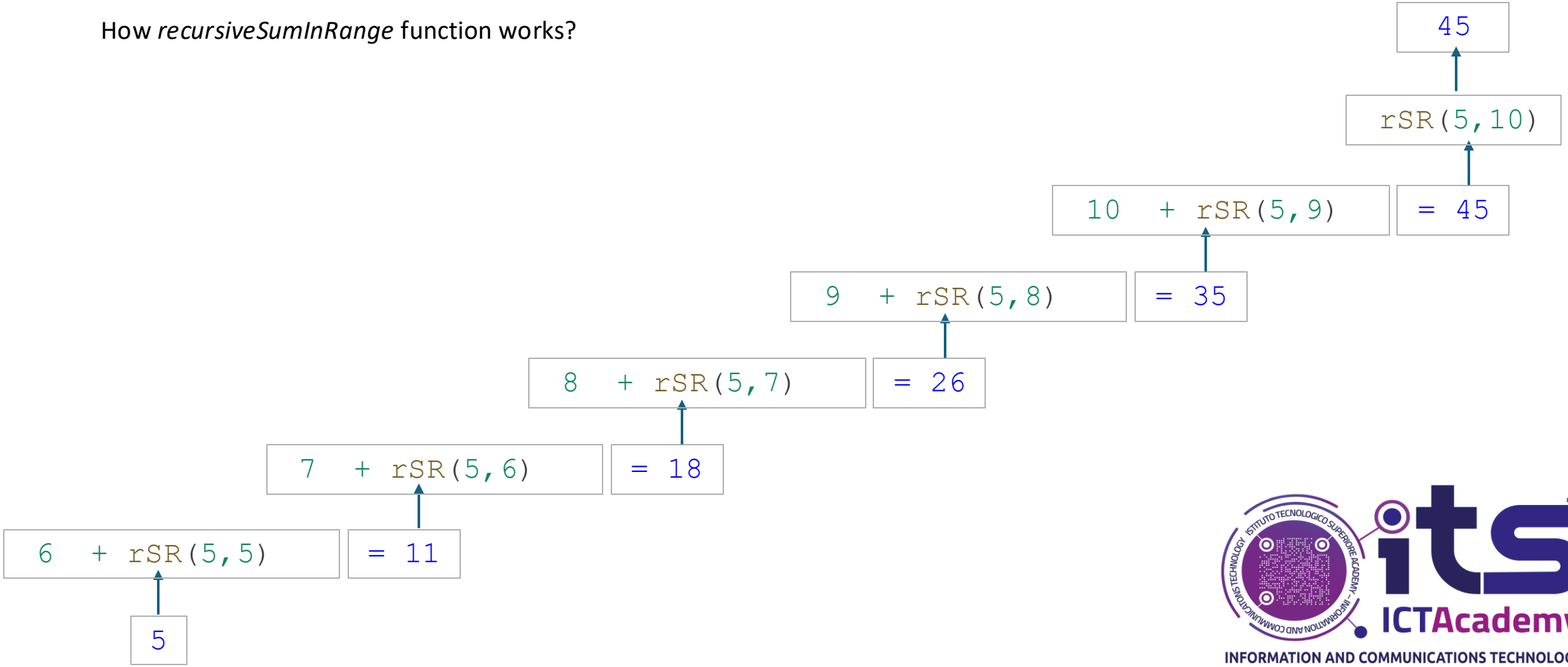
How *recursiveSumInRange* function works?

```
10  + rSR(5,9)   = 45
9   + rSR(5,8)   = 35
8   + rSR(5,7)   = 26
7   + rSR(5,6)   = 18
6   + rSR(5,5)   = 11
5
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 2- Recursive/6

How *recursiveSumInRange* function works?



ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Example 2- Recursive/7

Testing the *recursiveSumInRange* function.

```
# calling sumInRange function for a=5, b=10
print(recursiveSumInRange(5,10))

>>>
45
```

```
# calling sumInRange function for a=10, b=5
print(recursiveSumInRange(10,5))

>>>
45
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

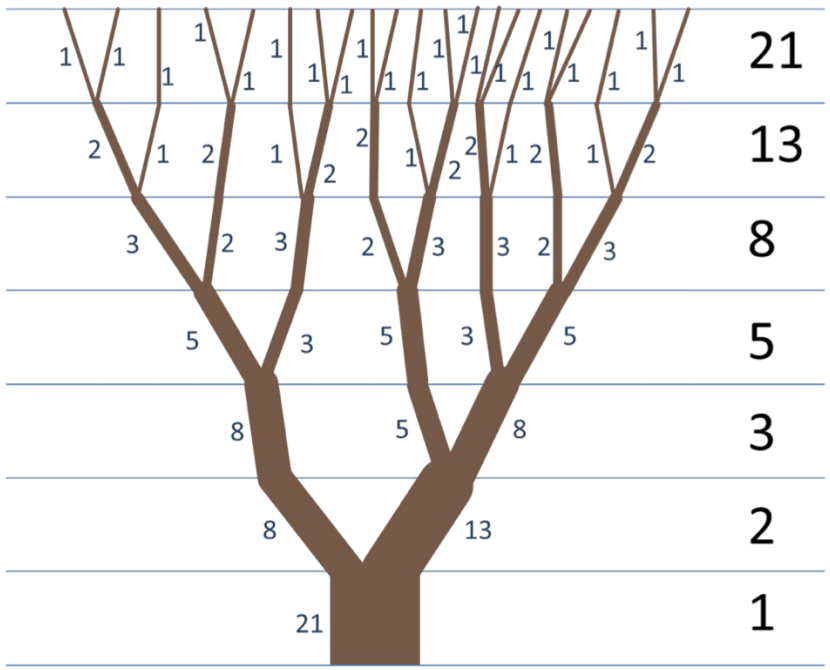
Recursive Functions – Fibonacci/1

What is the Fibonacci sequence?

It is a sequence of **integer** numbers in which each number is the sum of the two preceding ones.

Fibonacci sequences appear often in nature:

- Branching in trees
- Arrangement of leaves on a stem



its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Fibonacci/2

Without defining a Python code, compute Fibonacci series *for n = 6*.

Formal definition

$$F(n) = \begin{cases} 0 & if \quad n = 0 \\ 1 & if \quad n = 1 \\ F(n-1) + F(n-2) & if \quad n > 1 \end{cases}$$

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Fibonacci/3

```
F(6)
```

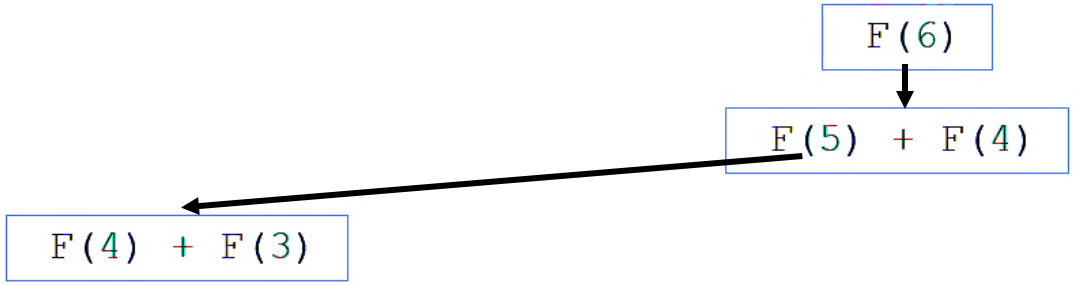
ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Fibonacci/3

```
F(6)
↓
F(5) + F(4)
```

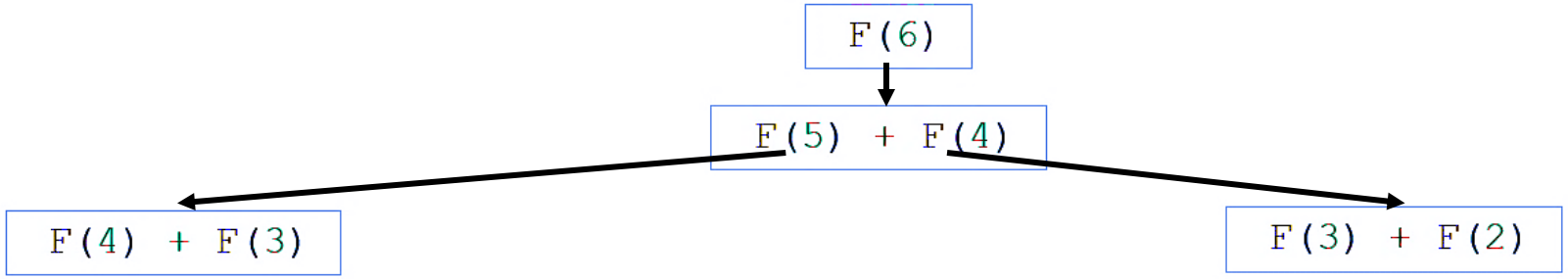
ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Fibonacci/3



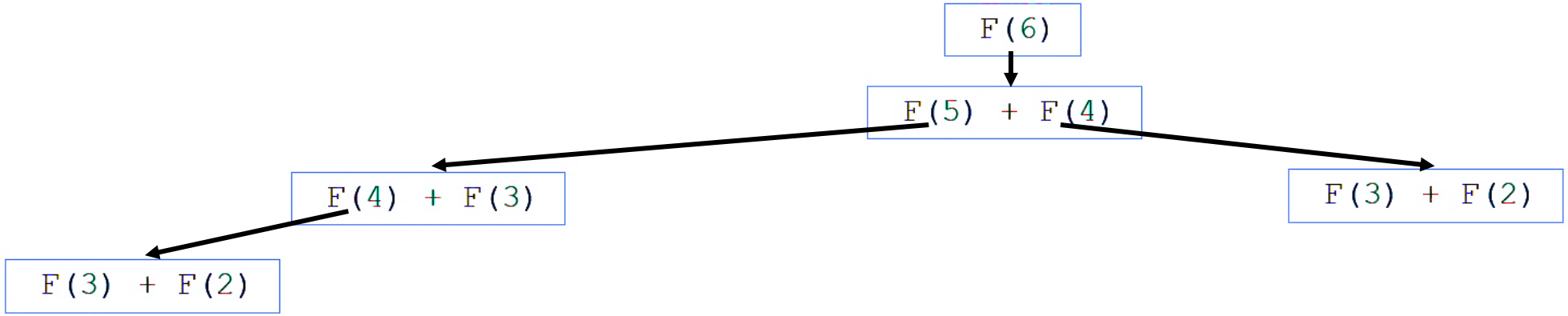
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Fibonacci/3

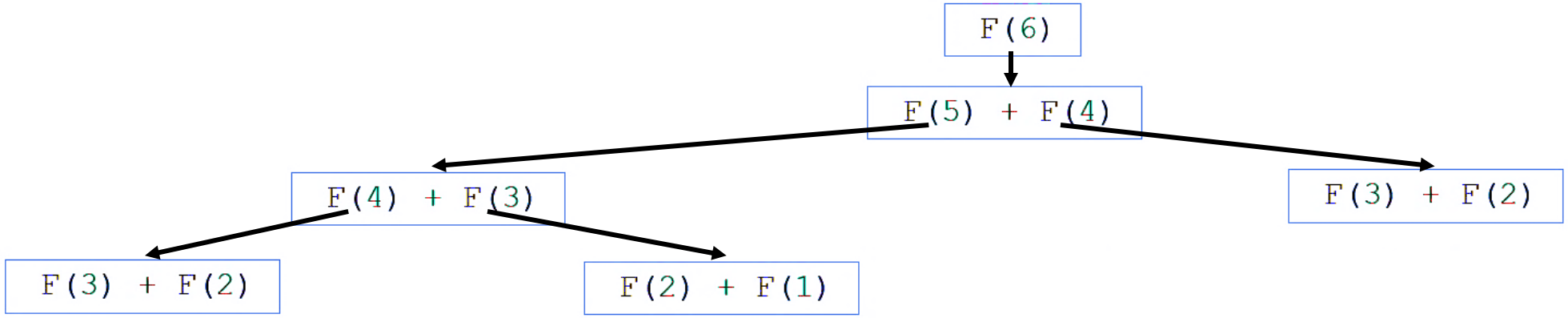


ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

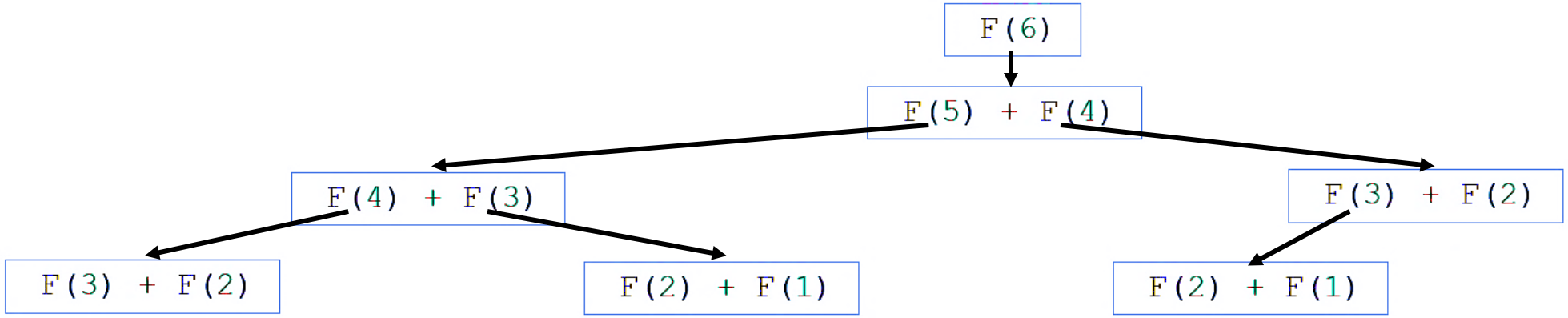
Recursive Functions – Fibonacci/3



Recursive Functions – Fibonacci/3

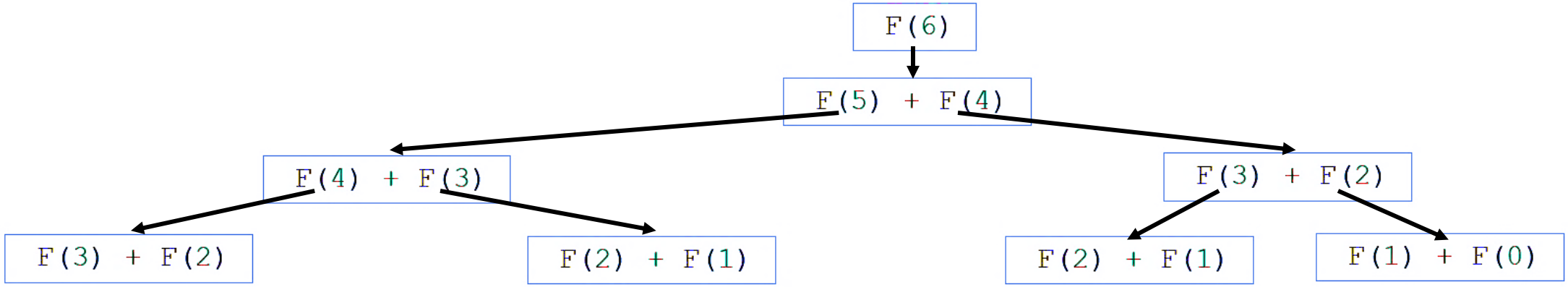


Recursive Functions – Fibonacci/3

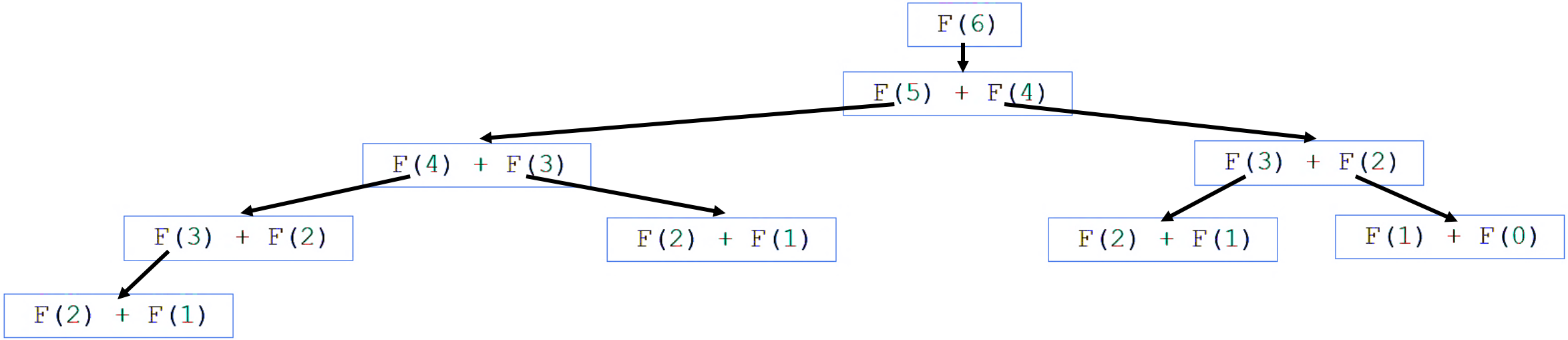


ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Fibonacci/3

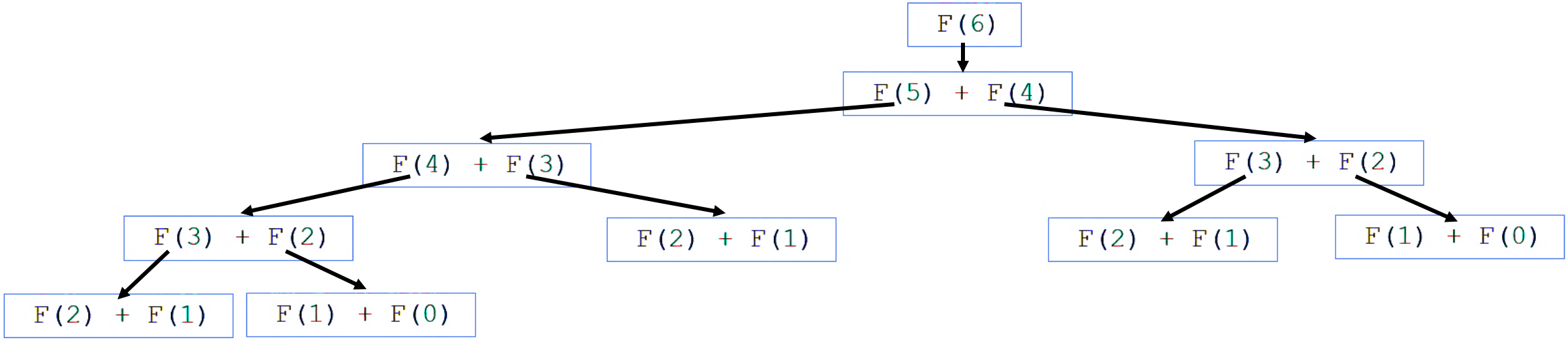


Recursive Functions – Fibonacci/3



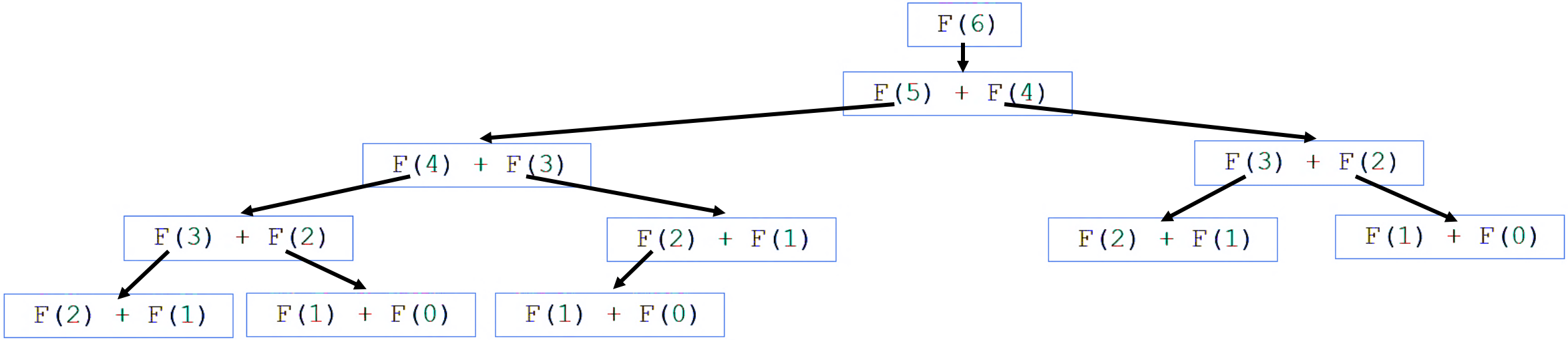
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Fibonacci/3

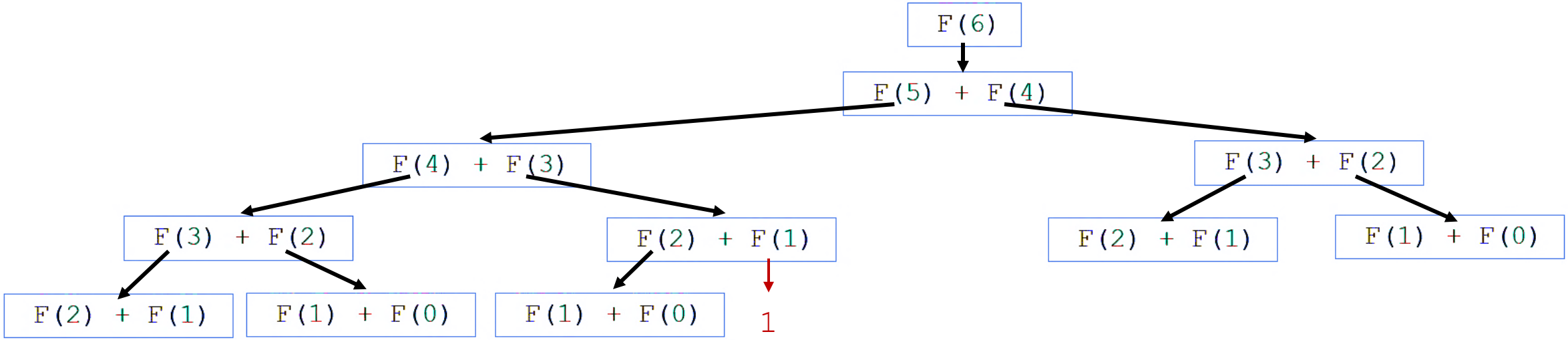


ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Fibonacci/3

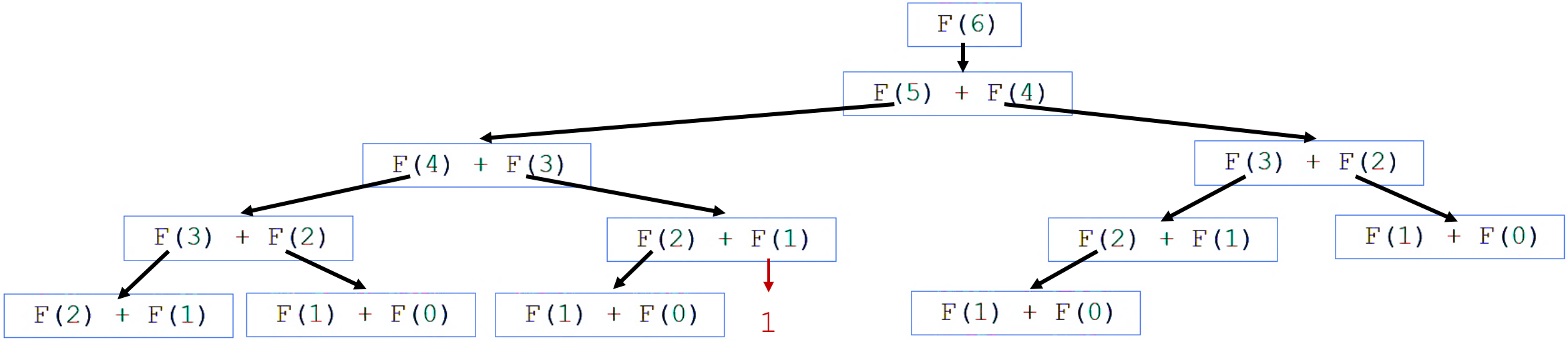


Recursive Functions – Fibonacci/3

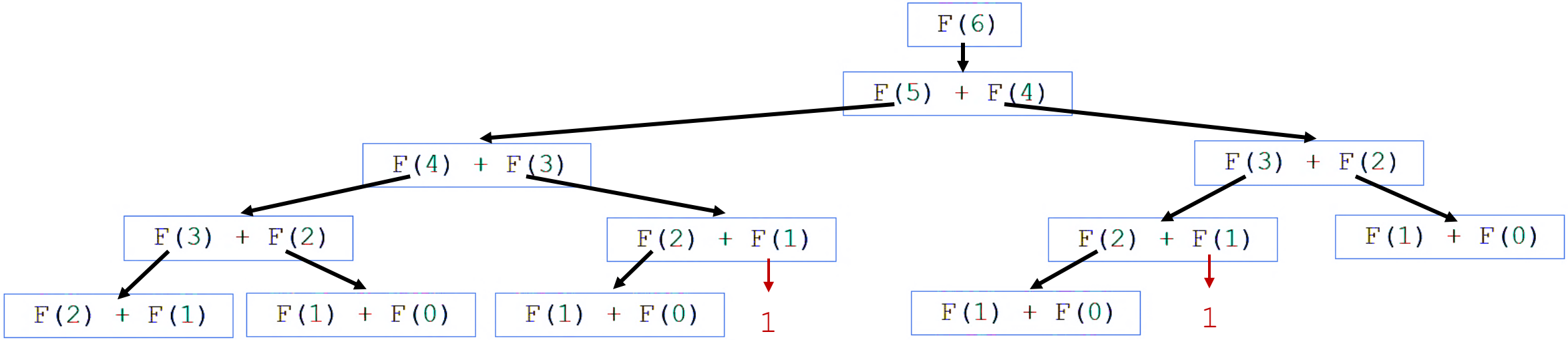


ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

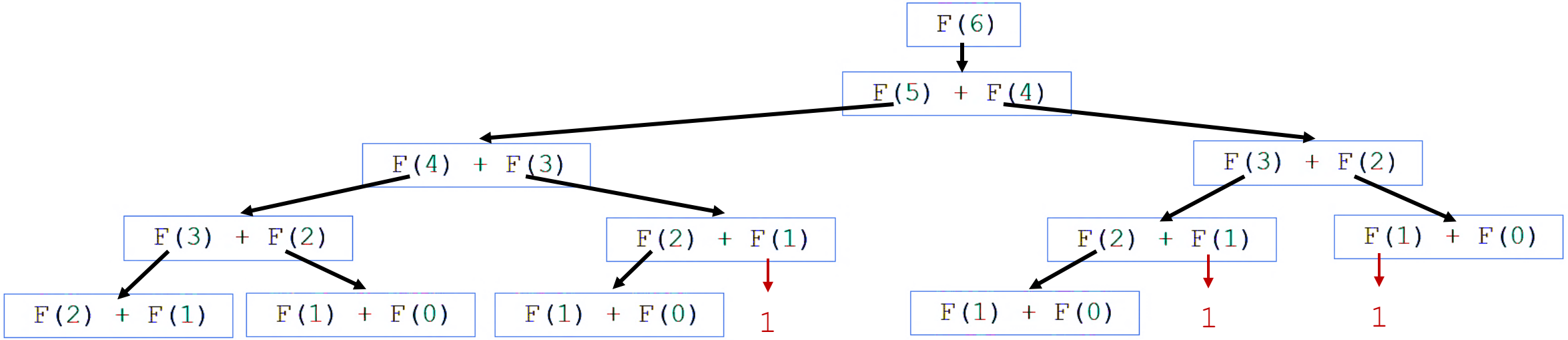
Recursive Functions – Fibonacci/3



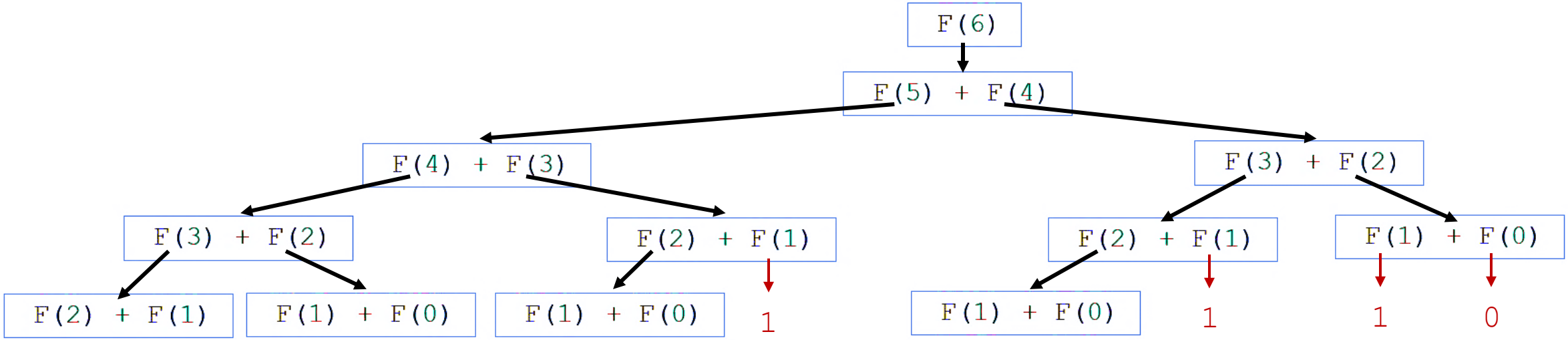
Recursive Functions – Fibonacci/3



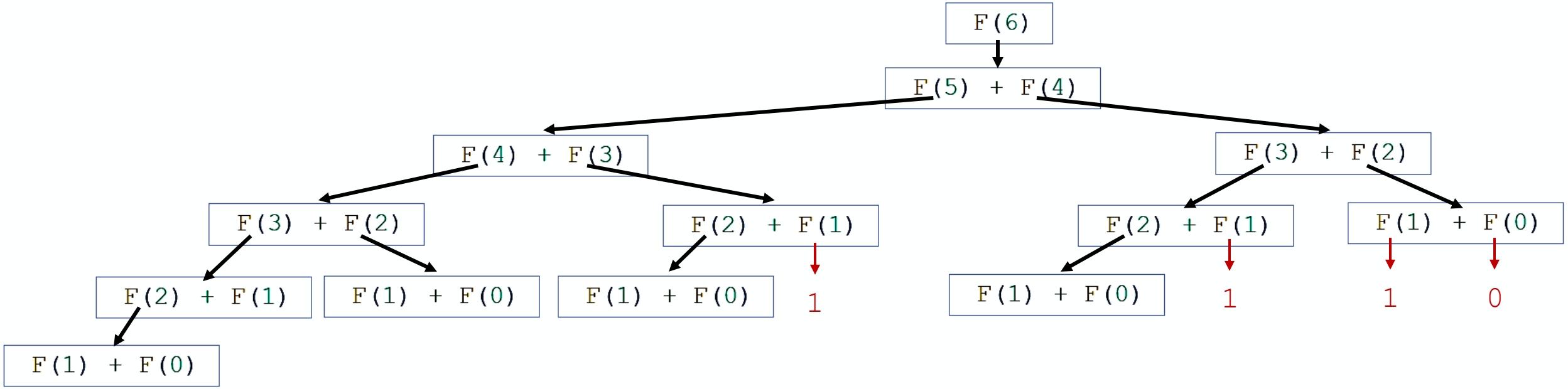
Recursive Functions – Fibonacci/3



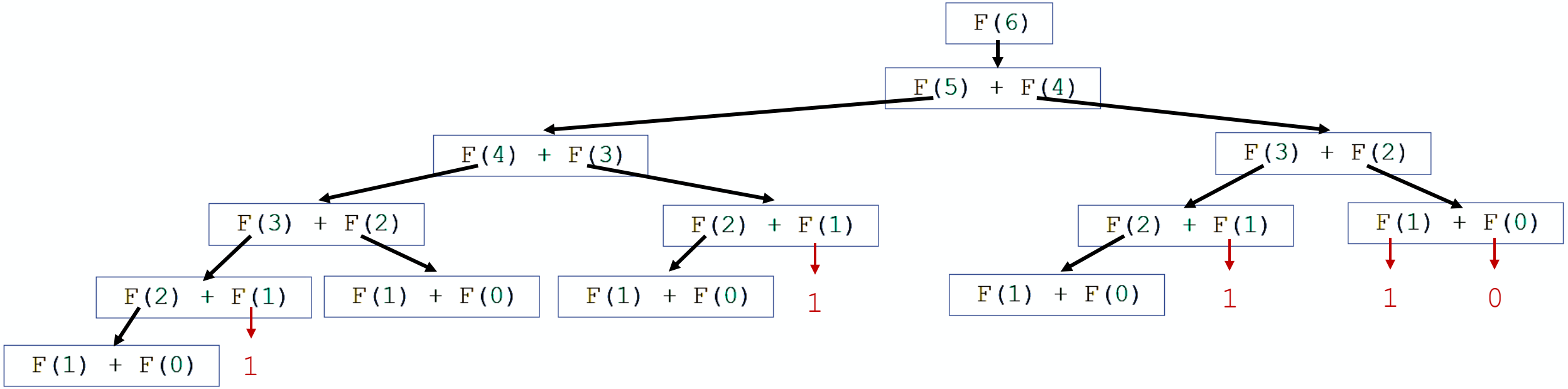
Recursive Functions – Fibonacci/3



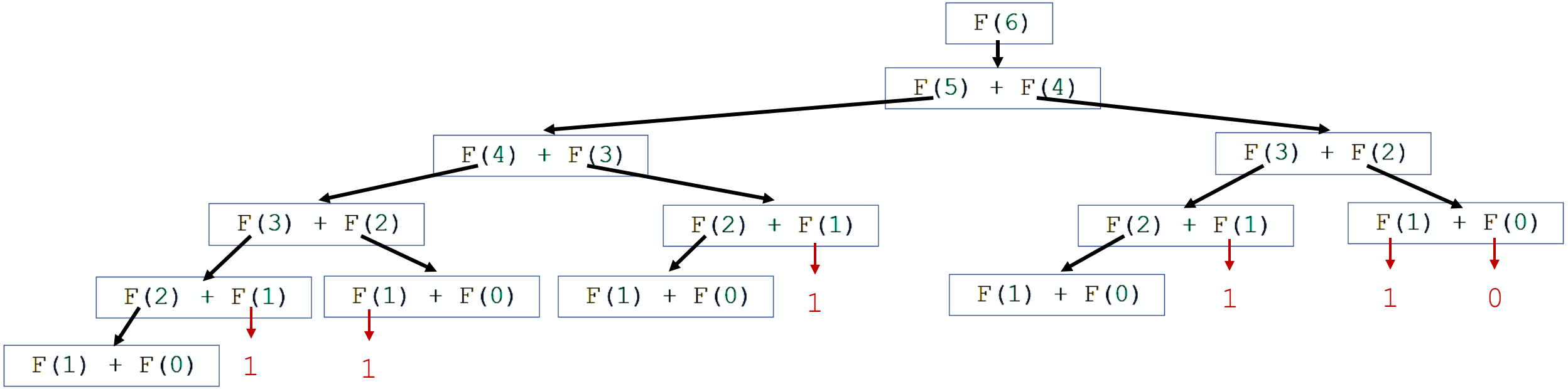
Recursive Functions – Fibonacci/3



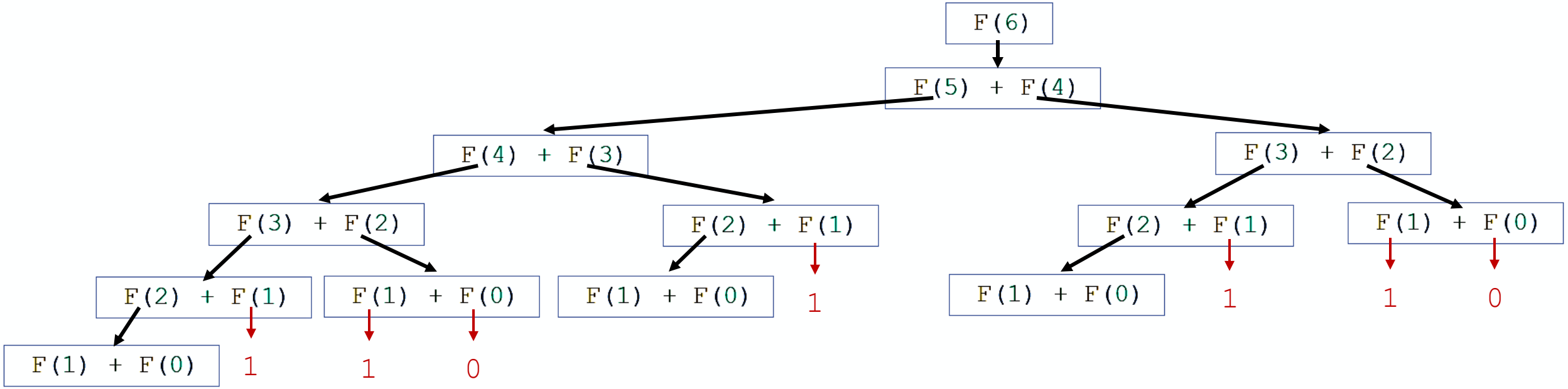
Recursive Functions – Fibonacci/3



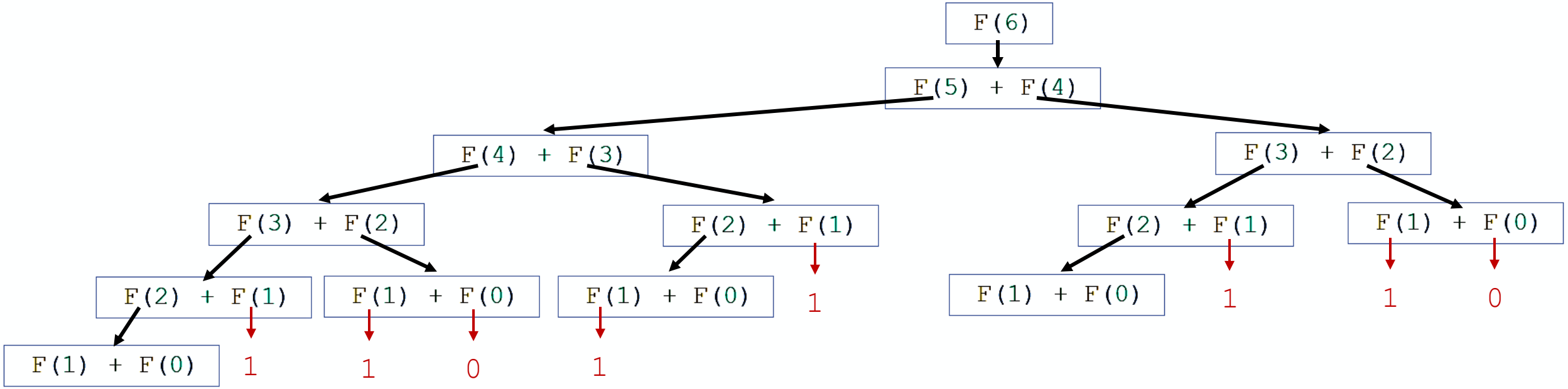
Recursive Functions – Fibonacci/3



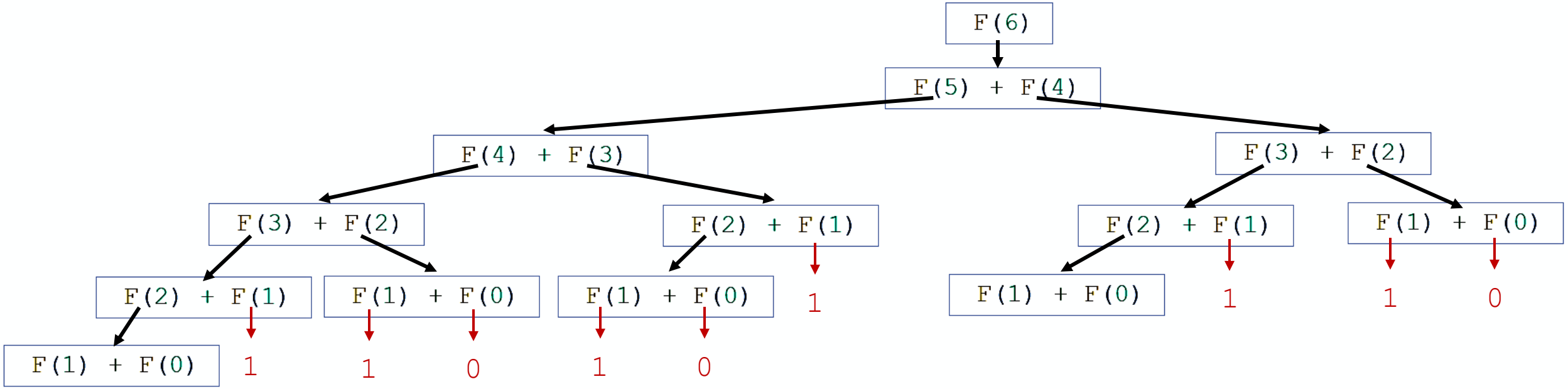
Recursive Functions – Fibonacci/3



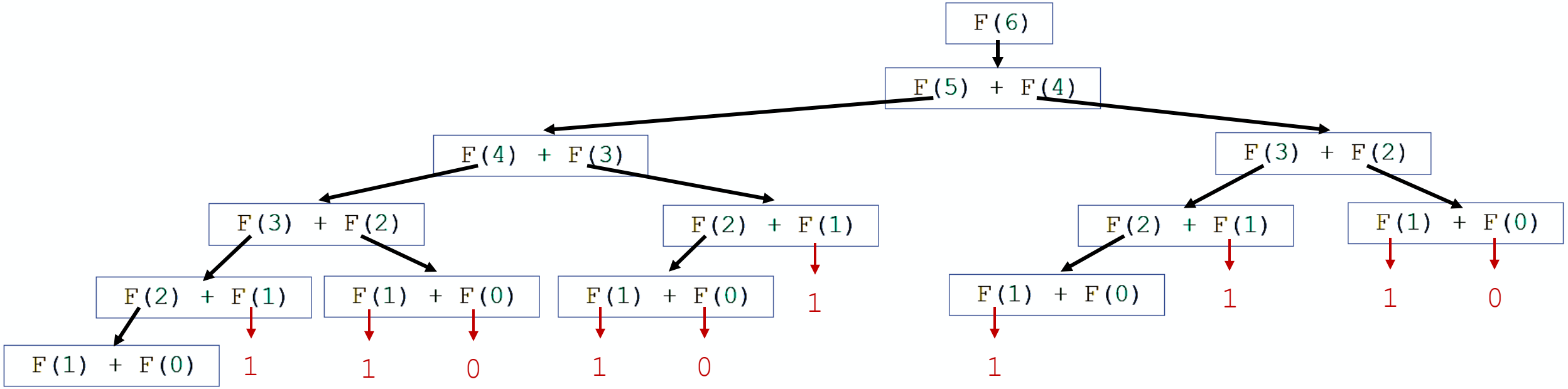
Recursive Functions – Fibonacci/3



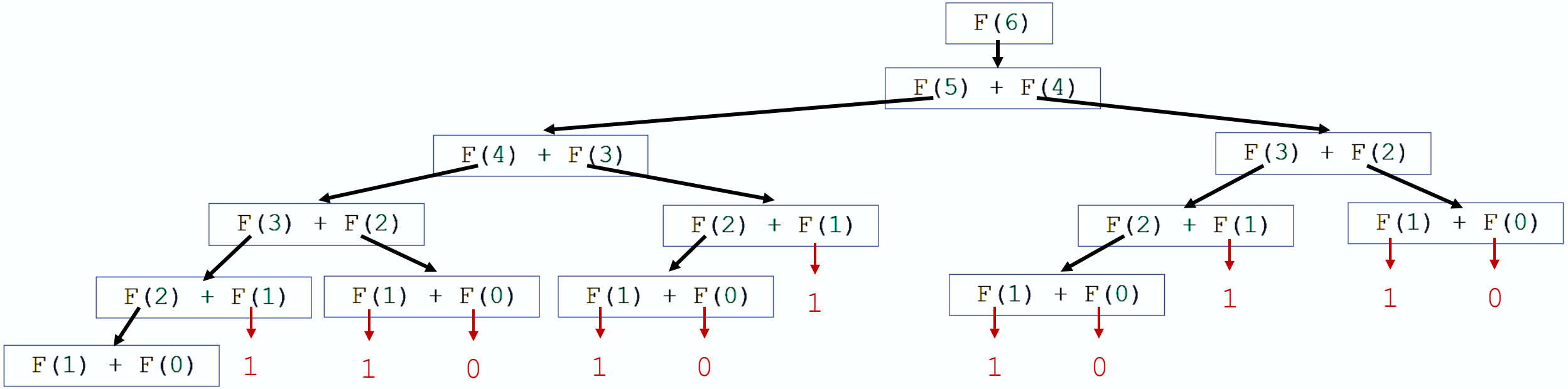
Recursive Functions – Fibonacci/3



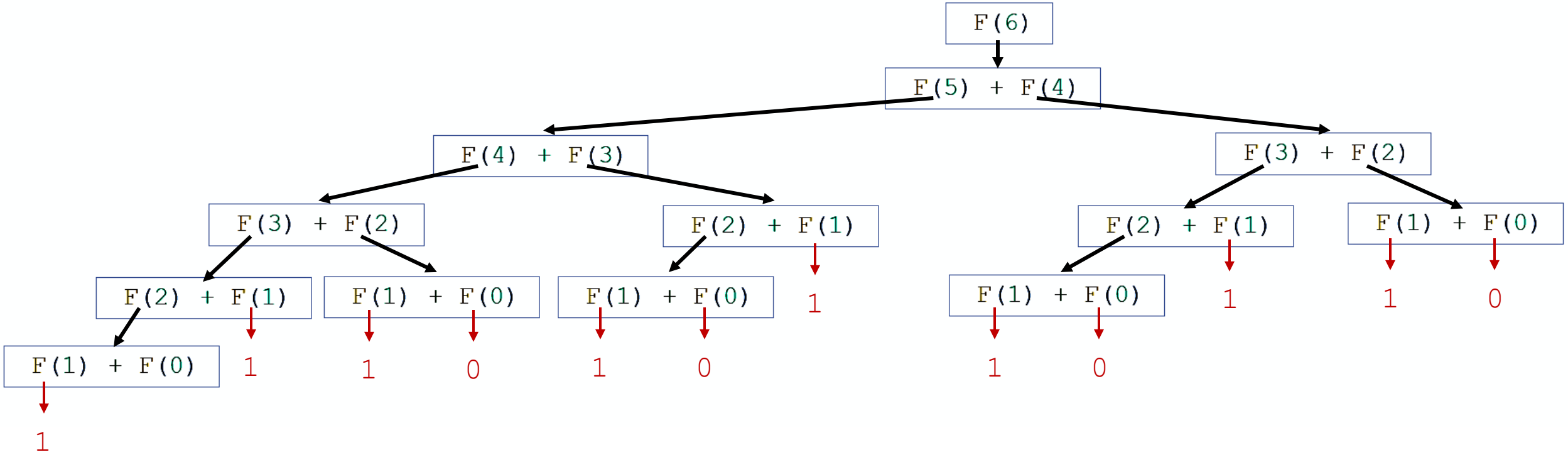
Recursive Functions – Fibonacci/3



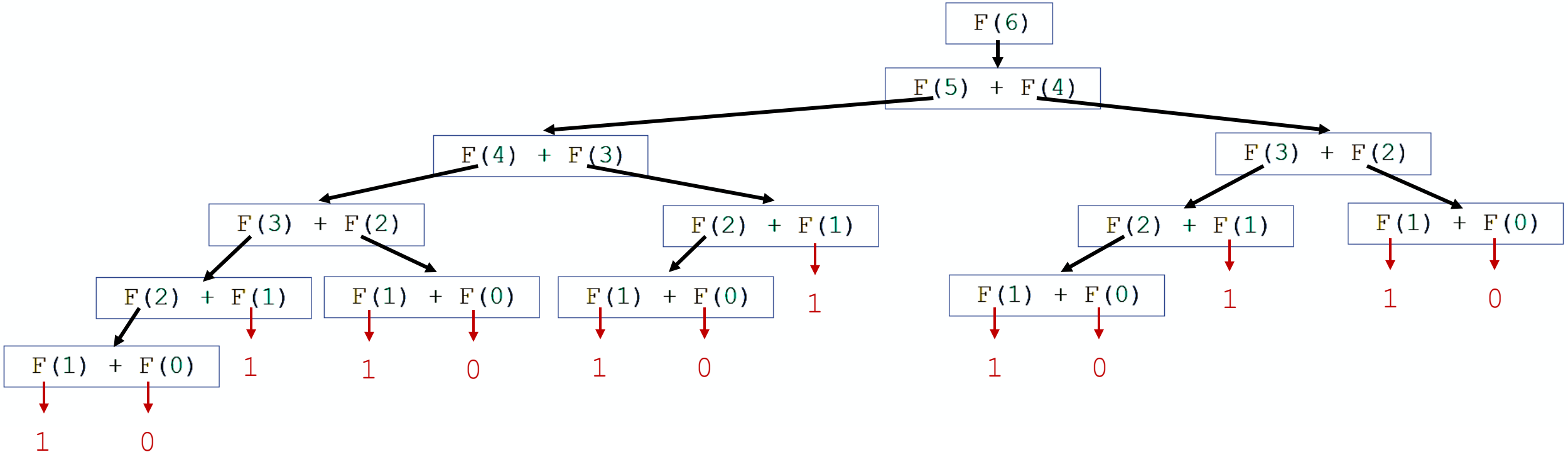
Recursive Functions – Fibonacci/3



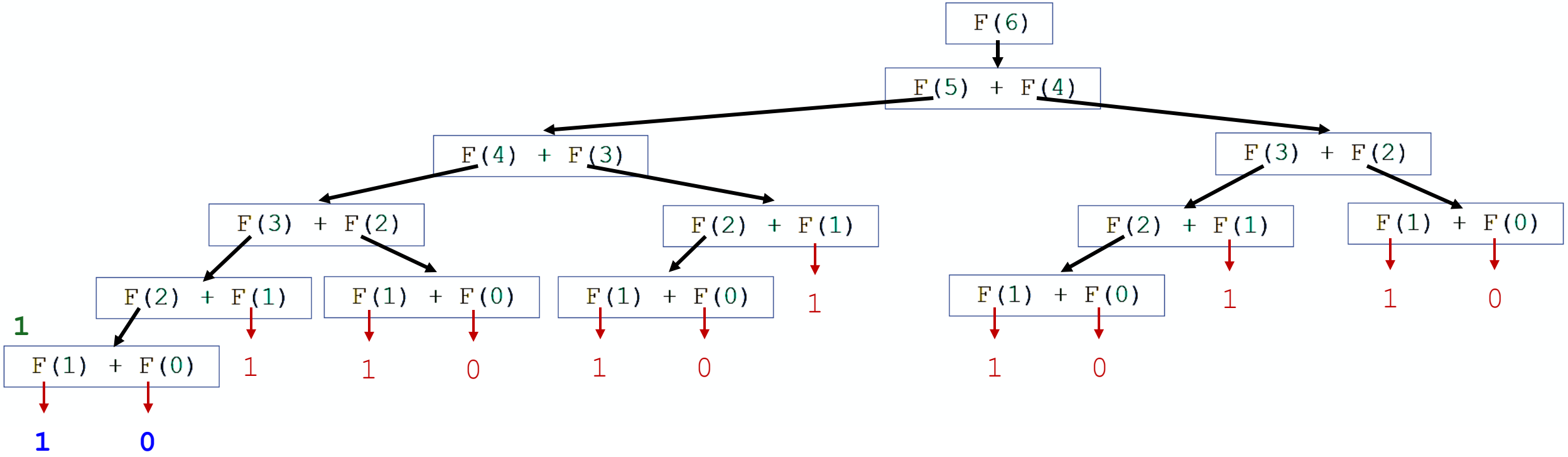
Recursive Functions – Fibonacci/3



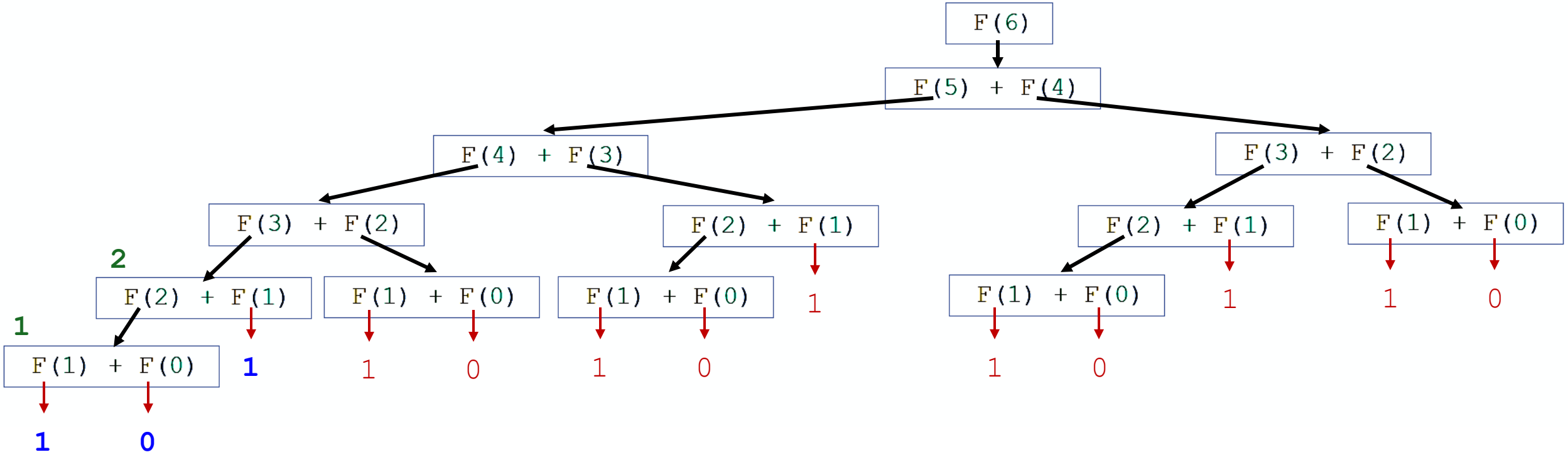
Recursive Functions – Fibonacci/3



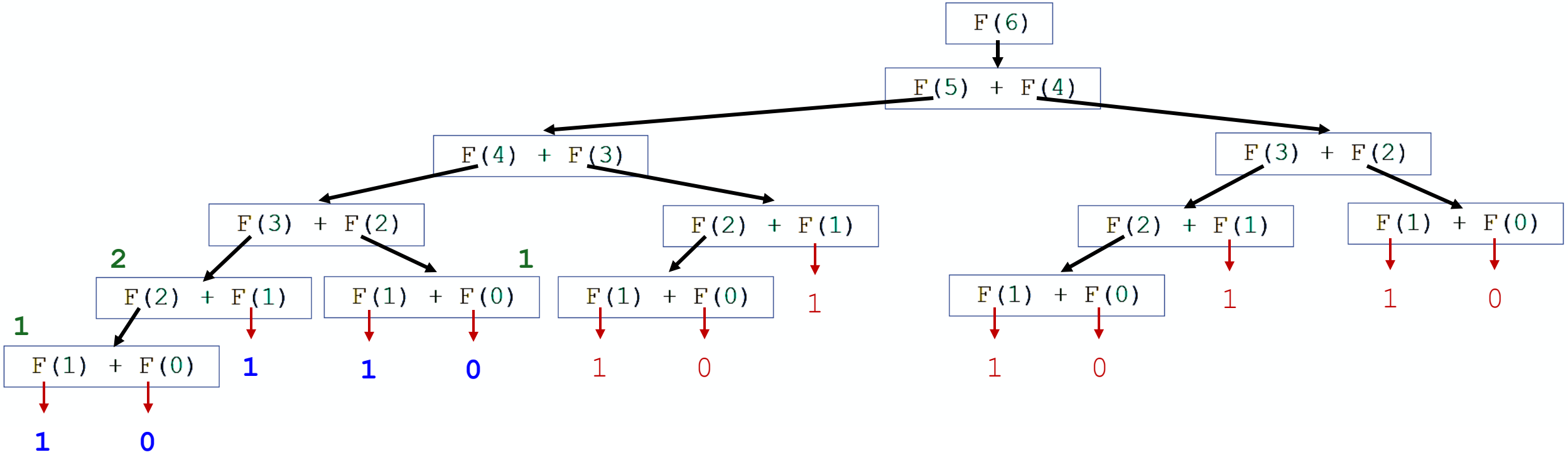
Recursive Functions – Fibonacci/3



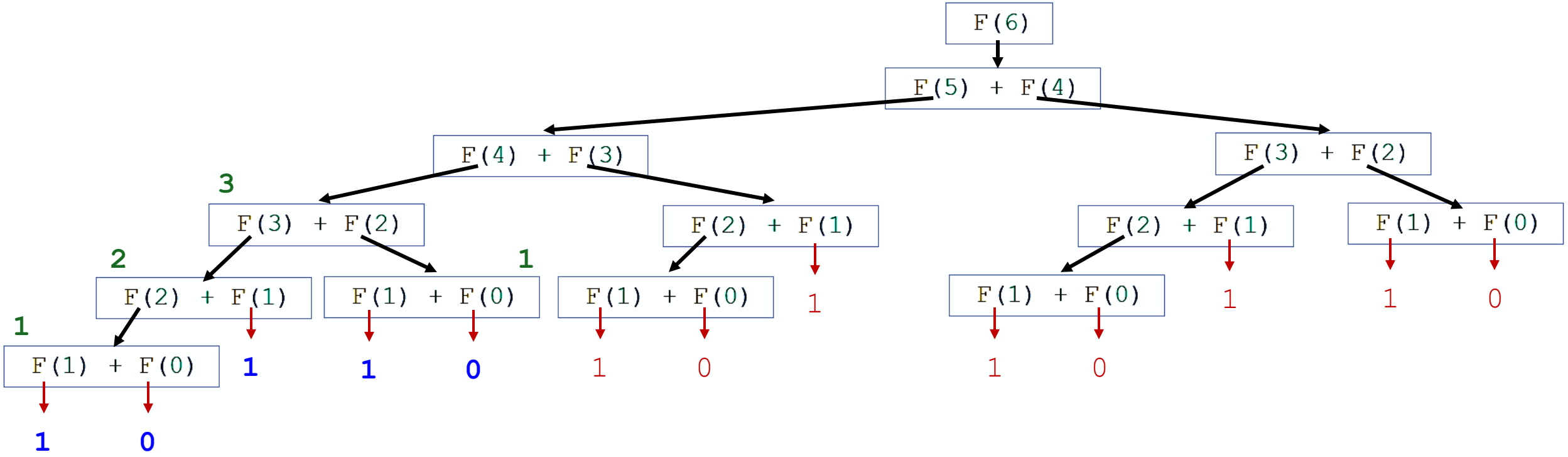
Recursive Functions – Fibonacci/3



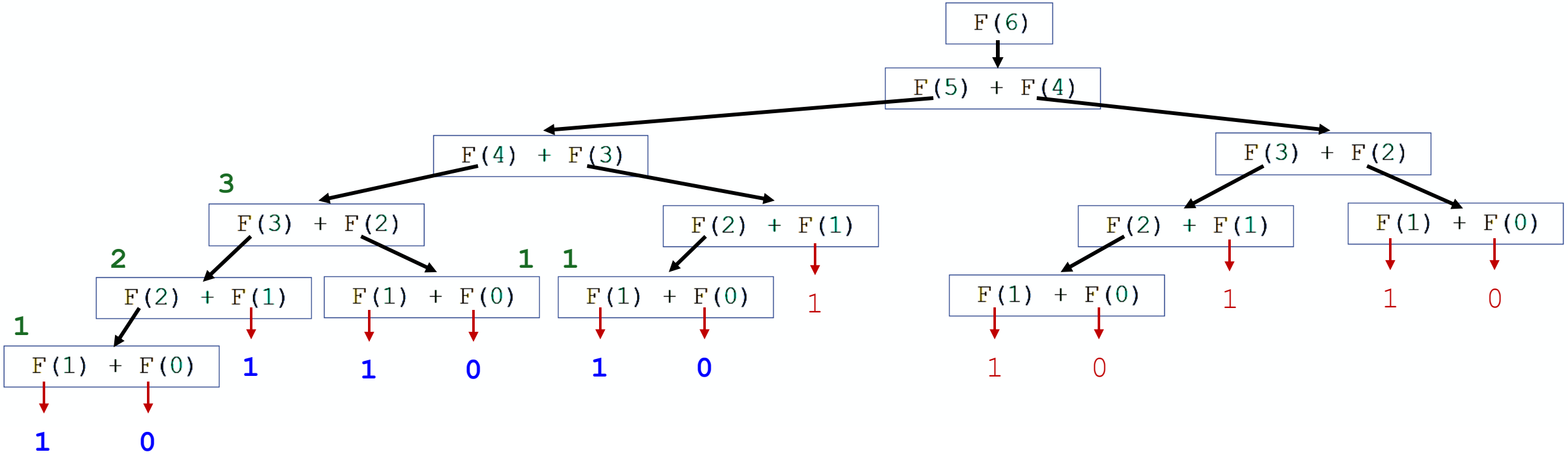
Recursive Functions – Fibonacci/3



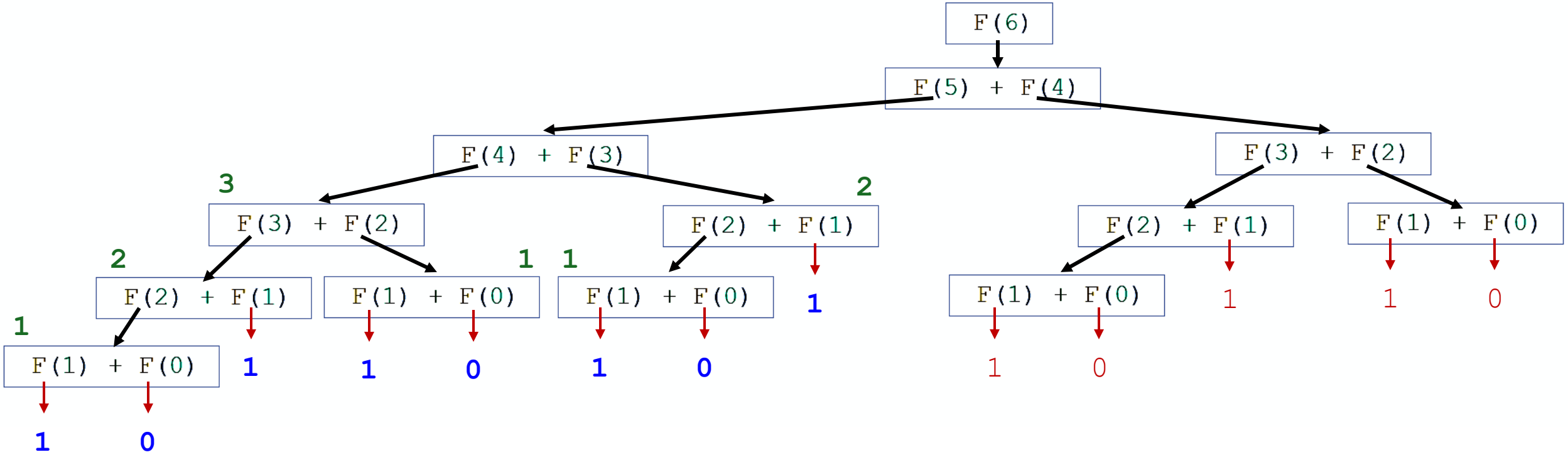
Recursive Functions – Fibonacci/3



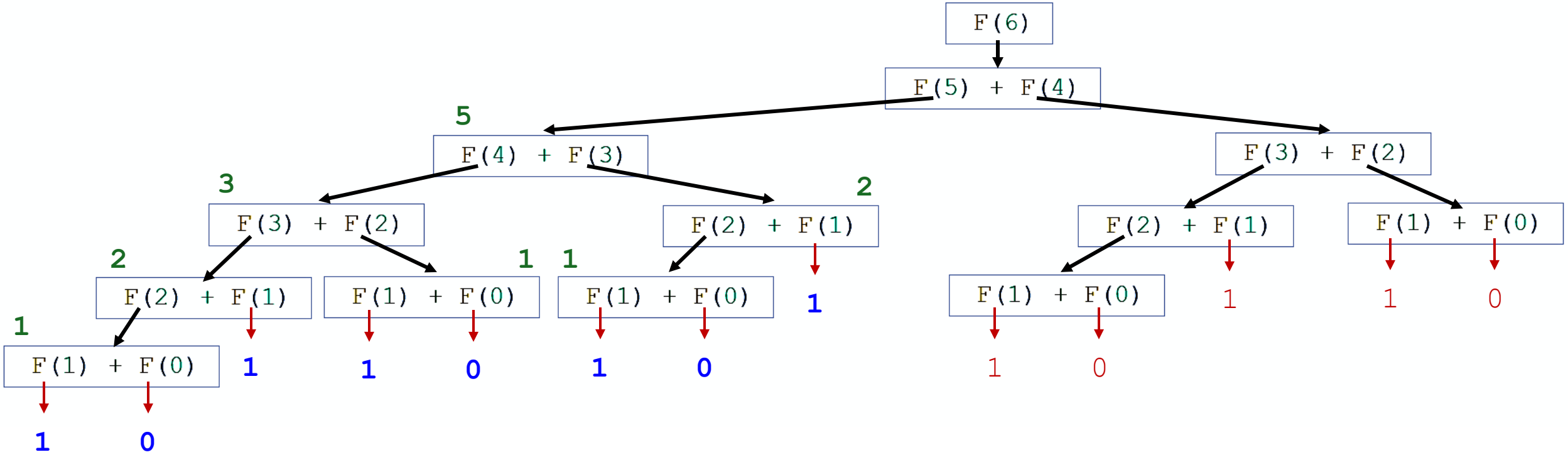
Recursive Functions – Fibonacci/3



Recursive Functions – Fibonacci/3

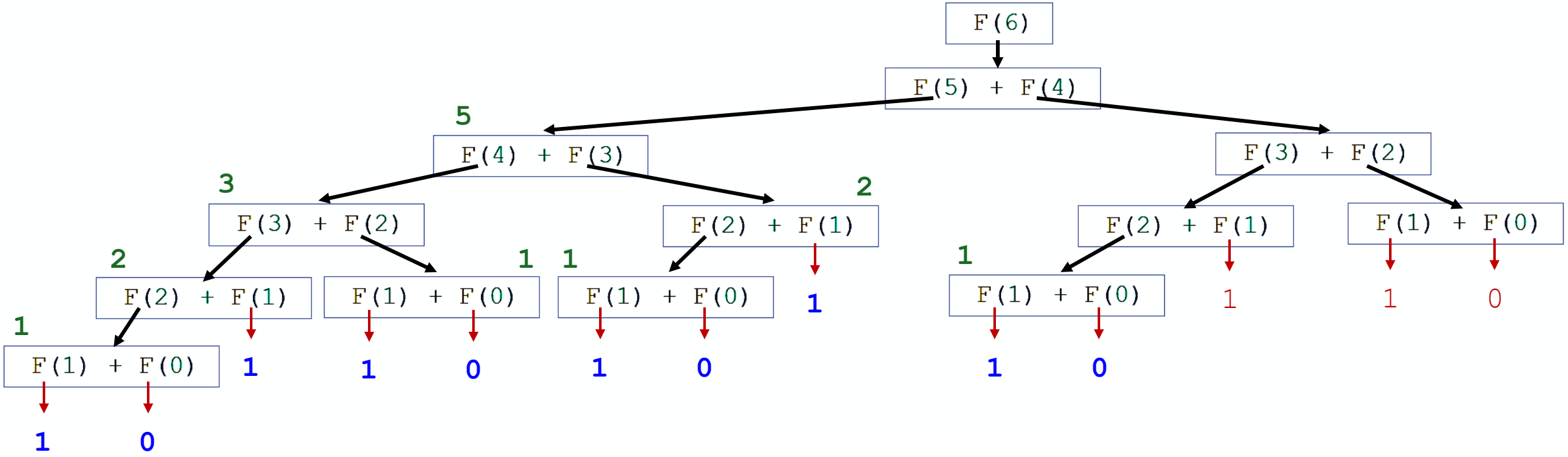


Recursive Functions – Fibonacci/3

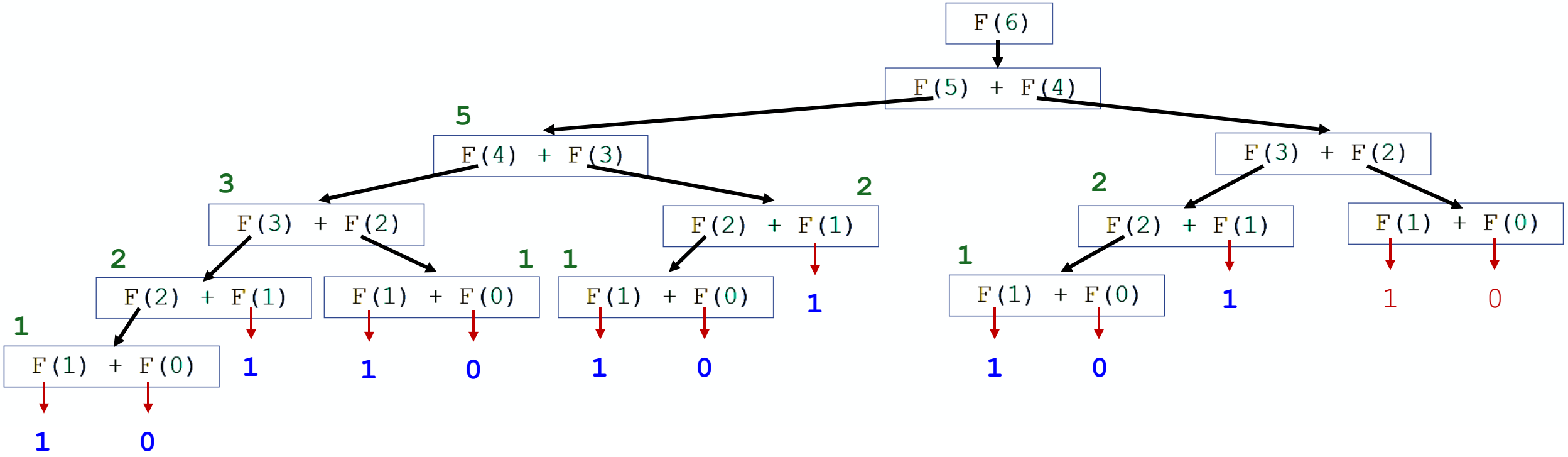


ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

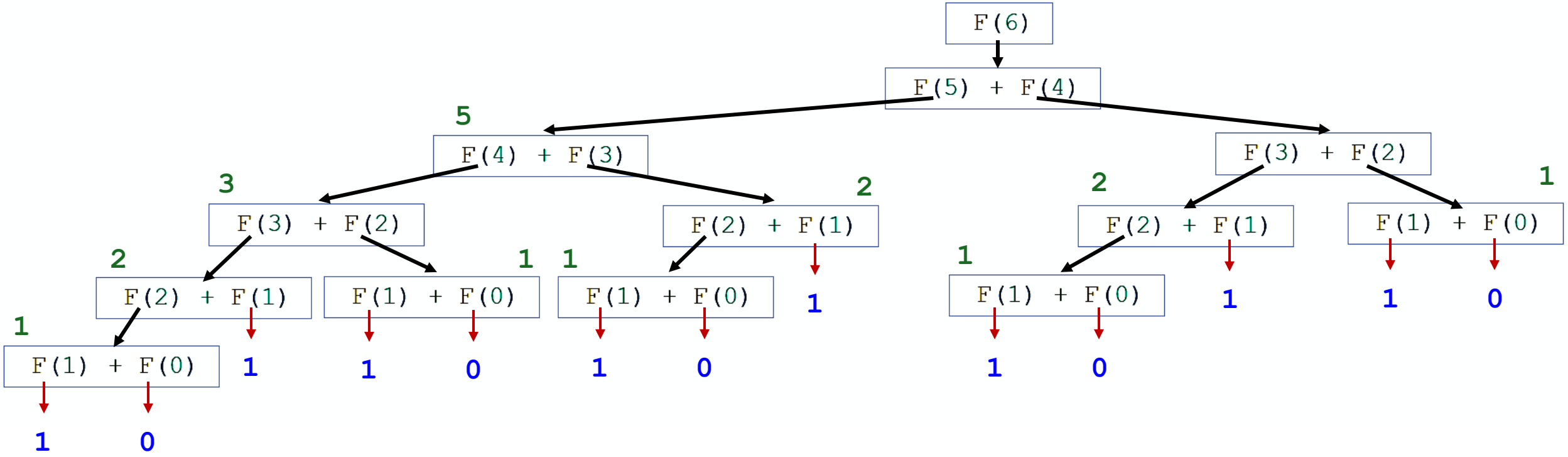
Recursive Functions – Fibonacci/3



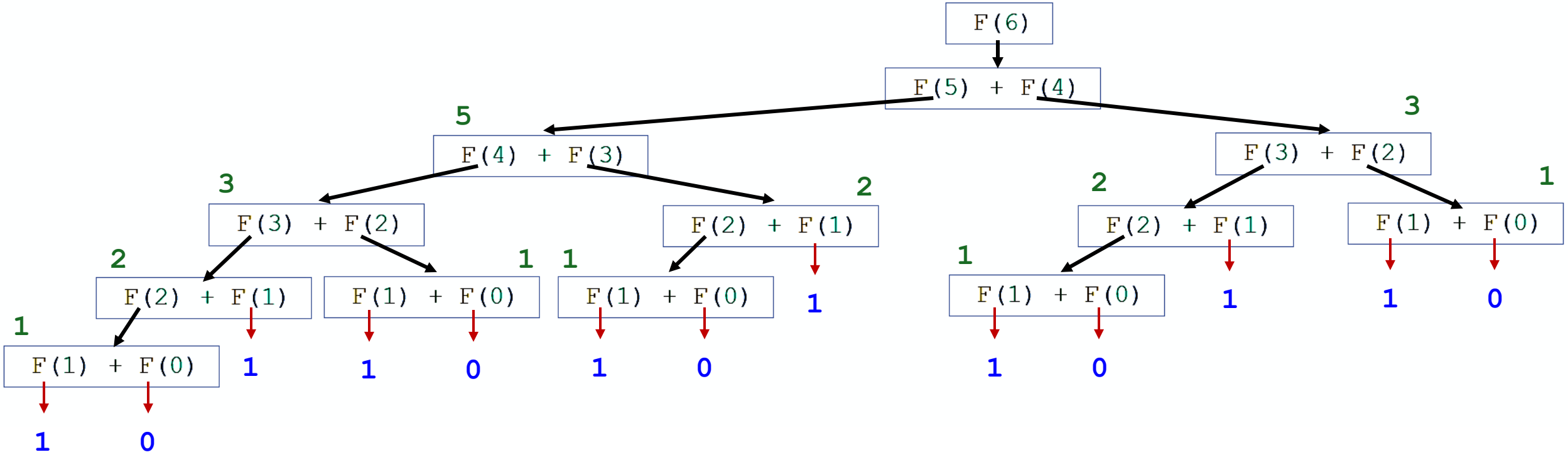
Recursive Functions – Fibonacci/3



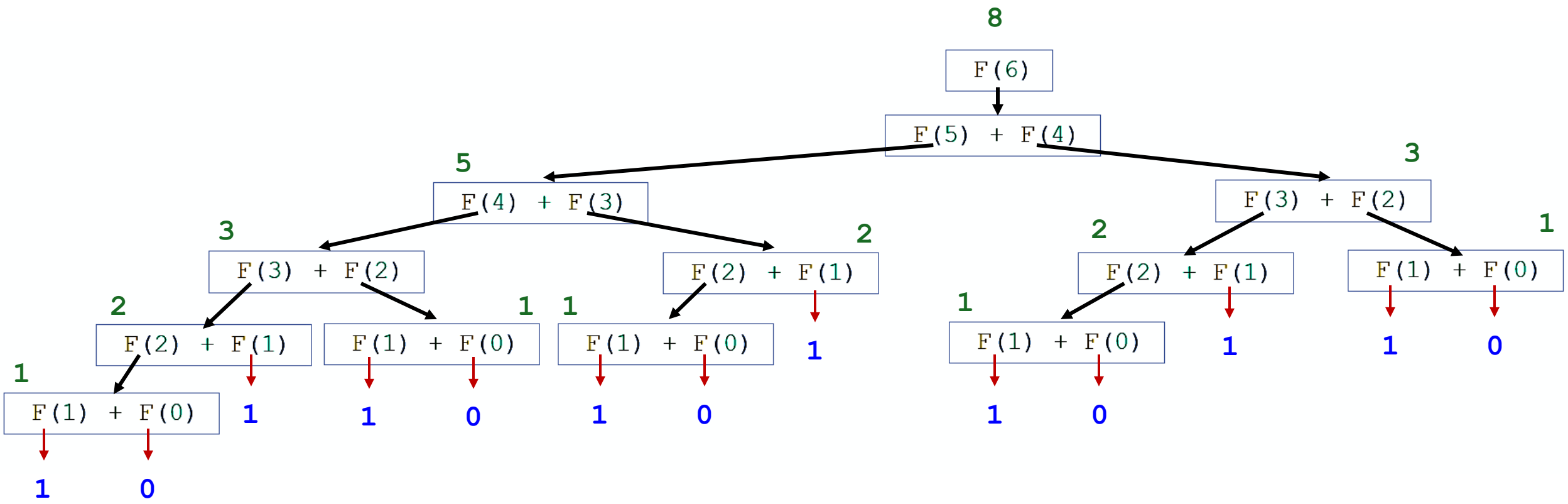
Recursive Functions – Fibonacci/3



Recursive Functions – Fibonacci/3



Recursive Functions – Fibonacci/3



Recursive Functions – Fibonacci/2

write a recursive function called *fibonacci* that calculates the Fibonacci series *for n = 6*.

Expected Output:

8

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Fibonacci/3

First, let's consider the simplest case of the problem to be solved.

Based on how the function is defined,

- if *n* is less than or equal to 0, it should return the value 0.
- If *n* is equal to 1, it should return the value 1.

```
def fibonacci(n:int) -> int:

    # n is negative or 0
    if n <= 0:

        return 0


    # n = 1
    elif n == 1:

        return 1
```

Formal definition

$$F(n) = \begin{cases} 0 & if \quad n = 0 \\ 1 & if \quad n = 1 \\ F(n-1) + F(n-2) & if \quad n > 1 \end{cases}$$

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Fibonacci/4

If *n > 1*, the definition of the Fibonacci series tells us that the other terms can be calculated using the formula

- Fibonacci(n-1) + Fibonacci(n-2)

```
def fibonacci(n:int) -> int:

    # n is negative or 0
    if n <= 0:

        return 0


    # n = 1
    elif n == 1:

        return 1


    # otherwise, compute Fibonacci series applying its formula
    else:

        return int(fibonacci(n-1) + fibonacci(n-2))
```

Formal definition

$$F(n) = \begin{cases} 0 & if \quad n = 0 \\ 1 & if \quad n = 1 \\ F(n-1) + F(n-2) & if \quad n > 1 \end{cases}$$

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Fibonacci/5

If *n > 1*, the definition of the Fibonacci series tells us that the other terms can be calculated using the formula

- Fibonacci(n-1) + Fibonacci(n-2)

```
def fibonacci(n:int) -> int:

    # n is negative or 0
    if n <= 0:

        return 0


    # n = 1
    elif n == 1:

        return 1


    # otherwise, compute Fibonacci series applying its formula
    else:

        return int(fibonacci(n-1) + fibonacci(n-2))
```

Formal definition

$$F(n) = \begin{cases} 0 & if \quad n = 0 \\ 1 & if \quad n = 1 \\ F(n-1) + F(n-2) & if \quad n > 1 \end{cases}$$

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Recursive Functions – Fibonacci/6

Testing the recursive *fibonacci* function for *n = 6*.

```
print(fibonacci(6))
```

```
>>>
8
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY – INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY